AI-Assisted Coding: From Idea to Working Prototype

VS Code, GitHub Copilot and three POCs with Streamlit, FastAPI, SQLite & XGBoost

Prof. Dr. Christoph Weisser

AI-Driven Automation • Bielefeld School of Business, HSBI

Winter Semester 2026/27

Slides, notebooks and data: github.com/ChrisW09/Python-for-AI-Driven-Automation

About your lecturer



Prof. Dr. rer. pol. Christoph Weisser

MLitt (St Andrews), MSc (Oxford)

- Professor of Mathematics, in particular Business Data Science, at HSBI
- Former Technical Lead for Artificial Intelligence and Industrial PhD Supervisor at BASF
- **Research:** Data Science, Machine Learning and Artificial Intelligence
- **Focus:** application of AI in business and industry

christoph.weisser@hsbi.de

github.com/ChrisW09/Python-for-AI-Driven-Automation

About your lecturer

Introduce yourself briefly (one minute). The message for this deck: in industry, prototypes decide which automation ideas get funded. Whoever can turn an idea into a small working app within a day has a real advantage in any business function.

Make clear that nobody needs prior programming experience. What the students need is curiosity, precise thinking and the discipline to check results – exactly what business students are trained for.

Optional ice-breaker: "Who has ever written a line of code? Who has used ChatGPT or a similar tool to get code?" The mix of answers shows why the course starts with a live demo instead of theory.

Your journey in this module

From a live demo to three working prototypes – and on to LLMs and agents

Session 1

- **1** Live demo — idea → app in ten minutes
- **2** Setup — GitHub, Copilot, Git, Python, VS Code

Session 2 and following

- **3** Method — Git, prompts, review
- **4** How apps work — frontend, backend, database
- **5** POC 1 — Streamlit data app
- **6** POC 2 — + FastAPI + SQLite
- **7** POC 3 — + XGBoost model

8 Wrap-up: from POC to production → **Next deck:** LLMs, RAG & Agentic AI

Course repository: `github.com/ChrisW09/Python-for-AI-Driven-Automation` – notebooks, exercises, data, slides

LLM access for your projects: we provide a **course API key** for smaller and larger LLMs – you will receive it **by email**. Treat it like a password: never pass it on or put it on GitHub.

Your journey in this module

Walk through the roadmap in 90 seconds: first you see what is possible, then you learn to do it yourself. Session 1 is the live demo plus the complete setup on the students' own laptops – it takes time, and a clean setup saves hours later. From Session 2 on: working method, how apps are built, then three POCs that build on each other. The LLM deck explains the technology behind the tools and adds three LLM prototypes.

Organisational hints: (1) Show the course repository on GitHub – notebooks from Python basics to AI engineering, exercises with solutions, data and slides; students new to Python start with its onboarding and foundations modules. (2) Announce the course API key: you send it by email; it gives access to smaller and larger LLMs and may be used in the students' own projects. It is a password: keep it in a local `.env` file, never in code or on GitHub, never outside the course – and report a leak at once, so it can be replaced for everyone.

Learning objectives

After this part of the module you can . . .

- set up a professional toolchain: **VS Code, GitHub Copilot, Git, Python**
- use the Copilot **agent** to build small apps from a precise description
- build a **Streamlit** data app on your own
- explain and implement a **three-tier architecture** (Streamlit + FastAPI + SQLite)
- build an ML application end to end: data → model → API → UI
- **read, test and improve** AI-generated code instead of trusting it blindly
- run a clean **Git/GitHub workflow** for your prototypes

The goal is not to become a software engineer – it is to turn business ideas into working prototypes you understand.

Learning objectives

Read the objectives aloud and connect them to the students' future roles: product owner, consultant, controller, marketing analyst. In all of these roles, being able to build a quick prototype – and to judge whether it works – changes how you work with IT and data teams.

Emphasise the sixth objective: reading and testing AI-generated code is the skill that distinguishes a responsible user from someone who just clicks "accept". We will practise it in every proof of concept.

Ask: "Which of these objectives sounds most difficult to you?" Usually the answer is the three-tier architecture – reassure them that Part 4 introduces every concept step by step.

What this course is *not*

Deliberately out of scope – so we can focus on what matters for prototypes

✗ Not a software-engineering course

No unit-test suites, no CI/CD pipelines, no production deployment.

✗ Not a frontend course

React, Vue or Tailwind appear only as an outlook.

✗ Not an ML theory course

XGBoost is applied and evaluated, not derived mathematically.

✗ Not a DevOps course

Docker, Kubernetes and cloud deployment come later – if at all.

We build *small, running* prototypes that answer a question – not products.

What this course is *not*

Managing expectations early avoids frustration. Some students will worry that they are expected to become developers; others will want to go much deeper. Both groups need to hear the same message: we build proofs of concept, and a POC deliberately leaves out many things a product needs.

At the end of the deck ("From POC to production") we list exactly what would be missing for a real product – that list is the bridge to professional IT projects.

Tip: mention that students who want to go deeper can use the extension challenges at the end of each POC.

Contents

1. From idea to app in ten minutes
2. Setting up your toolchain
3. Working with a coding agent
4. How apps are built
5. POC 1 – A data app with Streamlit
6. POC 2 – Frontend, backend and database
7. POC 3 – Adding a machine-learning model
8. Wrap-up

Contents

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 1 • Session 1: From idea to app in ten minutes

A live demo first – the theory comes afterwards.

- ✔ See what a coding agent can build from one precise description
- ✔ Understand where you, the human, stay in charge
- ✔ Know the difference between "vibe coding" and AI-assisted coding

Outline

Let's build something – live

A small automation for a finance team: the expense-claim checker

Today: a manual routine

Every month, a controller checks a few hundred expense claims line by line against the travel policy.

Goal of the demo

An app that does the *first check*: upload claims, see flagged items with a reason, download the list.

Rules the app should check

Rule	Example
Category limit	meals > 60 €, taxi > 80 €, hotel > 180 € per night
Weekend	claim dated on a Saturday or Sunday
Duplicate	same employee, date and amount twice

Limits are illustrative, not a real policy.

Pick a boring, rule-based check – the perfect candidate for a first automation prototype.

This is the hook of the whole module. Do the demo live on the projector, before students have installed anything. Explain the scenario in business terms first: a repetitive control task, clear rules, a spreadsheet as input – many students will know such tasks from internships.

Keep the rules simple and visible on the slide so that everybody can later check whether the generated app really implements them. That check is the real lesson: we will not just admire the result, we will verify it.

Timing: plan about 15 minutes for the demo block (this slide to "What just happened?") and about 30 minutes for the whole of Part 1, so that the setup still gets about 55 minutes of a 90-minute Session 1. Preparation tip: have an empty folder open in VS Code, Copilot signed in, Agent selected, permissions on Manual. Have a pre-built backup version in a second folder in case the network is slow.

The one prompt

Everything the agent needs – goal, data, rules, output, deliverables

Demo prompt for Copilot – Agent (typed live)

Build a small Streamlit app `app.py` that automates the first check of employee expense claims. First create `sample_data.py`, which generates about 300 realistic demo claims (employee, department, date, category, amount in EUR, description) with some problems. The app loads the demo data or an uploaded CSV and flags claims that (1) exceed the category limit (meals 60, taxi 80, hotel 180 EUR per night), (2) fall on a weekend, or (3) look like duplicates (same employee, date and amount). Show KPIs, a table of flagged claims with the reason, a bar chart of flagged amounts per department and a CSV download of the flagged claims. Add `requirements.txt` and a short README, create a virtual environment `.venv`, install the requirements there and start the app.

Goal & data

what to build, demo data

Rules & output

testable rules, concrete UI

Deliverables

files, packages, run it

The one prompt

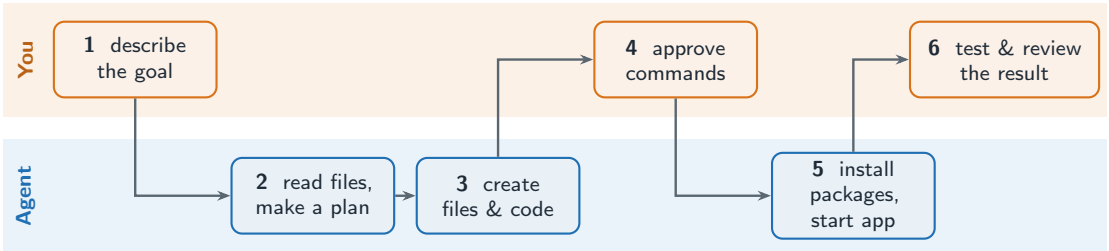
Type or paste this prompt live into the Copilot chat (Agent selected). While the agent works, narrate what you see: it reads the empty workspace, proposes a plan, creates files one by one and then asks for permission to run `pip install` and `streamlit run`.

Gloss the jargon in one breath: Streamlit turns a Python script into a web app, `requirements.txt` lists the packages the app needs, and `.venv` is the project's private Python – all three return in Parts 2 and 3. Point out the structure of the prompt – it already follows the anatomy of a good coding prompt that we discuss in Part 3: goal, demo data, testable rules, concrete output, deliverables, and the instruction to run it.

Timing: the generation usually takes one to three minutes. If the agent asks questions, answer them aloud so the students hear the dialogue. If something fails, even better: show how you paste the error message back into the chat.

What just happened?

You decided, the agent executed – and you checked



The agent's tools

read and edit files • run terminal commands • read error messages and try again

Your job

a precise goal • conscious approvals • checking that the rules are *really* implemented

What just happened?

Debrief the demo with this diagram. The orange boxes are the moments where a human decides; the blue boxes are the agent's work. The agent is not magic: it is a language model that can use tools – reading and writing files and running terminal commands – in a loop until the task looks done. The LLM deck explains this mechanism in detail.

Now test the rules together: open the flagged table, find a hotel claim above 180 euros, check a Saturday date, search for a duplicate. Ask the students: "How would you know if the duplicate rule is wrong?" This is exactly the review habit we want.

Mention that the agent asked before running commands because the permission level was set to Manual – we will keep it that way in class.

Why this matters for AI-driven automation

Test ideas cheaply

A clickable prototype in hours: learn early whether an automation idea is worth it.

The bottleneck shifts

From *typing code* to *specifying* and *verifying* – core business skills.

A shared language

A running prototype says more than twenty pages of requirements.

But: a prototype is not a product

Security, data protection, tests and operations still need professionals – see the wrap-up.

Business people who can prototype become better partners for IT – not replacements for it.

Why this matters for AI-driven automation

Connect the demo to the module theme. Automation projects often fail not because of technology, but because requirements are vague and nobody tests the idea early. A prototype makes the idea concrete: users can click through it, disagree with it, and improve it.

The second card is the key insight: when an agent can type the code, the scarce skills are precise specification and careful verification. Business students are well prepared for both – they know the process and the rules.

Balance the enthusiasm with the warning box. Ask: "What would be missing before the finance department could use this app with real employee data?" Collect answers such as access control, data protection, audit trail – and keep them for the wrap-up.

"Vibe coding" vs. AI-assisted coding

Same tools, very different attitude

> **Vibe coding** (Karpathy, 2025)
> "[. . .] fully give in to the vibes, embrace exponentials, and forget that the code even exists."

> **AI-assisted coding** (Willison, 2025)
> "[If you] reviewed it, tested it thoroughly and made sure you could explain how it works [. . .] that's not vibe coding, it's software development."

	Vibe coding	AI-assisted coding
You read and understand the changes	✗	✓
You run and test it, including an error case	✗	✓
Every working step is committed to Git	✗	✓

Our rule: you own every line you run or share – no matter who (or what) typed it.

Source: Karpathy (2025); Willison (2025)

"Vibe coding" vs. AI-assisted coding

Andrej Karpathy coined the term "vibe coding" in a post on X on 2 February 2025. He himself wrote that it was "not too bad for throwaway weekend projects". The term spread quickly and is now often used for any AI-supported programming – which is misleading.

Simon Willison drew the line clearly in March 2025: if you review, test and can explain the code, it is not vibe coding, it is software development. That is why this course speaks of **AI-assisted coding**. The third row mentions Git: for now, a commit is simply a saved snapshot of the project you can return to – Part 3 explains it properly.

Ask the students which column they would want their bank's developers to be in. The point is not to forbid fast experimentation, but to know which mode you are in – and to switch to the right-hand column as soon as real data or other people are involved.

What the evidence says

AI assistance is powerful – and uneven

↑ Well-defined tasks

In controlled experiments, people with an AI assistant finish clearly specified tasks much faster.

Peng et al. (2023); Noy & Zhang (2023)

↓ Large, familiar code

Experienced developers working in their own large projects were *slower* with AI tools – while believing they were faster.

Becker et al. (2025)

The jagged frontier

Inside the AI's capability frontier, quality rises; just outside it, people who rely on AI make more mistakes.

Dell'Acqua et al. (2026)

Using AI well is a skill: know when to delegate, when to check – and when to think for yourself.

Exact figures for questions: Peng et al. (2023): 95 professional developers wrote an HTTP server in JavaScript; the group with GitHub Copilot finished 55.8% faster (95% CI 21–89%). Noy & Zhang (2023, Science): 453 professionals, writing tasks with ChatGPT: 40% less time, 18% higher quality.

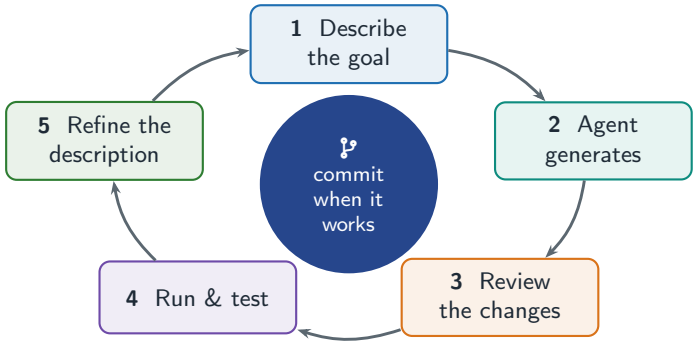
Becker et al. (2025, METR): 16 experienced open-source developers, 246 tasks in their own repositories. They expected AI to save 24% of the time and afterwards believed it had saved 20% – in fact they took 19% longer.

Dell'Acqua et al. (2026, Organization Science): 758 BCG consultants with GPT-4. Inside the frontier: 12.2% more tasks, 25.1% faster, about 32% higher quality. On a task outside the frontier, AI users were 19 percentage points less likely to be correct.

Update (METR blog, 24 Feb 2026; follow-up with 57 developers and 800+ tasks from late 2025): point estimates of 18% (returning) and 4% (new developers) *less* time with AI, but both confidence intervals include zero and METR calls the evidence very weak because of selection effects. Message: our POCs are well-defined tasks – ideal for AI assistance. But always verify.

The AI-assisted coding loop

Seconds to minutes per iteration – with a commit whenever something works



Small steps win: one feature per loop, test it, commit it – then the next one.

The AI-assisted coding loop

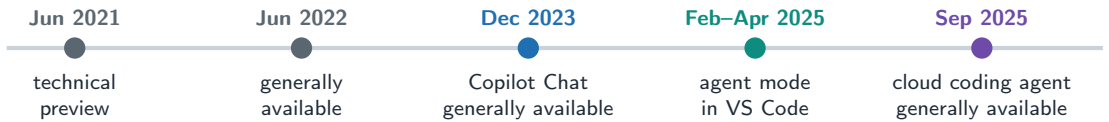
This loop replaces the classic "read documentation, learn syntax, write, debug, search the web" cycle. The big difference is speed: one iteration takes seconds to minutes instead of hours.

Stress steps 3 and 4 – they are where learning and quality happen. Reviewing means reading the diff the agent produced and asking it to explain anything unclear. Testing means running the app and trying at least one normal case and one error case.

The commit in the centre is the safety net: whenever the app works, commit. If the next iteration breaks something, you can always go back. Students who skip commits usually lose work in the first POC – say this explicitly.

From autocomplete to agents

Five years of GitHub Copilot



Gen. 1: autocomplete

AI completes the line or function you are typing.

Gen. 2: chat

AI answers questions, explains and rewrites code blocks.

Gen. 3: agents

AI plans, edits many files, runs commands and iterates – *with your approval*.

Dates, all from GitHub's own announcements: technical preview 29 June 2021; general availability 21 June 2022 (free for verified students from the start); Copilot Chat generally available 29 December 2023; agent mode preview in VS Code Insiders 6 February 2025 and available to all VS Code users in April 2025 (VS Code 1.99); the Copilot coding agent, which works on GitHub issues in the cloud and opens pull requests, became generally available on 25 September 2025 (now called "Copilot cloud agent").

The point for students: within four years the tool went from suggesting a line to autonomously changing a whole project. That is why the human role moves from typing to specifying and reviewing. In this course we use generation 3 – the agent in VS Code – but keep generations 1 and 2 in mind: they are still useful for small edits and questions.

What is a proof of concept (POC)?

And what it deliberately leaves out

Proof of concept

- a **running prototype** that proves one concrete idea
- small and focused – built in hours to days
- optimised for **learning speed**, not robustness
- just enough to answer one question – no more

Deliberately left out

tests, authentication, multiple users, caching, background jobs, deployment, visual polish

Typical POC questions

Can we detect this automatically? Is the data good enough? Would users accept this workflow?

Rule of thumb: better to prove three things quickly than to perfect one thing slowly.

What is a proof of concept (POC)?

Define the term precisely, because students will hear "POC", "prototype" and "MVP" used interchangeably in companies. A POC answers a question – technical feasibility, data quality, user acceptance. An MVP (minimum viable product) is already something real users use in production.

The warning box lists what we skip on purpose. This is not sloppiness: leaving these things out is what makes a POC fast. The danger in practice is when a POC is silently promoted to production without adding them.

Ask for an example from the expense-claim demo: which question did it answer? (Answer: can the three policy rules be checked automatically on our claim data?)

What we will build

Three POCs – each one adds architecture, nothing is taken away

POC 1 • Data app

Streamlit only

upload a CSV, descriptive statistics, charts

one process, one file

POC 2 • Three tiers

Streamlit + **FastAPI** + **SQLite**

same use case, professional structure

two processes, a REST API, a database

POC 3 • ML app

+ an **XGBoost** model

predict customer churn, log every prediction

data → model → API → UI



What we will build

Give the big picture before the setup, so students know why they install each tool. POC 1 is a single Streamlit app – the fastest way to a useful data tool. POC 2 solves exactly the same problem, but with a professional architecture: a separate backend (FastAPI) and a database (SQLite). Comparing POC 1 and POC 2 shows what architecture really means. POC 3 adds a machine-learning model and turns the app into a small ML product.

Each POC builds on the previous one, and nothing is taken away. By the end, students have seen all typical building blocks of a data-driven business application.

Transition: "To build these, you need a toolchain. Let's set it up – now, together."

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

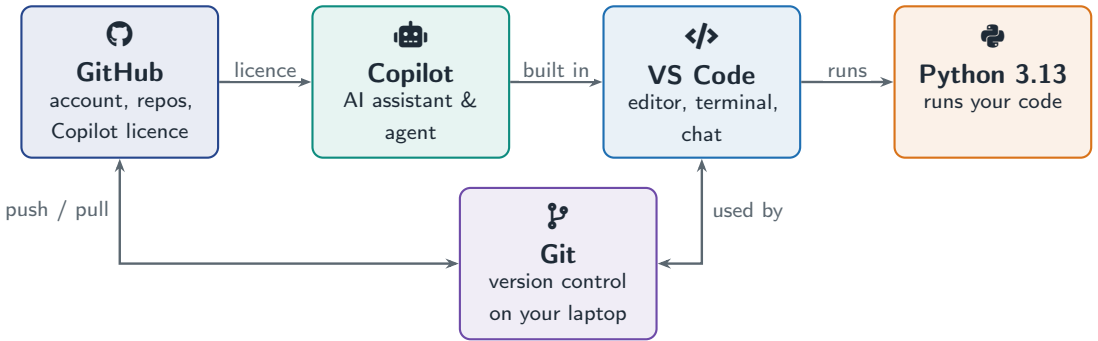
Part 2 • Session 1: Setting up your toolchain

Seven steps, in this order – we do them together in class.

- ✔ Create a GitHub account and apply for GitHub Education
- ✔ Install Git, Python 3.13 and VS Code
- ✔ Sign in to Copilot and run a first test
- ✔ Know the chat controls that keep you in charge

Your toolchain at a glance

Five tools, one job each



GitHub stores your code and your licence; VS Code is where you work; Git keeps every version.

Your toolchain at a glance

Give the mental model before installing anything. GitHub is a website: it hosts repositories (project folders with history) and manages the Copilot licence. Copilot is the AI – in VS Code it is built in, you only need to sign in. VS Code is the editor where you write, run and chat. Python is the language our prototypes use. Git is a program on your laptop that records versions; VS Code uses it, and it synchronises with GitHub.

Common confusion to address: "Git" and "GitHub" are not the same. Git is the tool, GitHub is a service built around it – like email and Gmail.

Timing: in a 90-minute Session 1 the front slides and Part 1 take about 35 minutes, so about 55 remain for the setup. That works if students did steps 1 and 2 before class (next slide): steps 3–7 take about 30–40 minutes plus troubleshooting, and whatever is unfinished becomes homework ("Setup done?").

Before class: start today

Verification takes days, not minutes

1 Create a GitHub account

`github.com/signup` – ideally with your HSBI email address; switch on two-factor authentication.

2 Apply for GitHub Education

Verification usually takes a few days – longer at the start of the semester. Apply as soon as your account exists.

No approval yet? No problem.

Every account can use **Copilot Free** at once. After approval, activate Copilot Student yourself (Education benefits → *Learn more*) – never pay.

Do both today – verification takes days, and Copilot Free bridges the gap.

This slide is meant to be sent out in advance (email or learning platform). The student verification by GitHub is the only step in the setup that students cannot speed up: GitHub's docs state that approval and Copilot activation are separate steps and that the student benefit "can take several days to finish applying after verification"; eligibility is re-evaluated every month (as of Sep 2026).

Background: in spring 2026 GitHub paused new sign-ups for individual Copilot plans for some weeks and reopened them gradually from June 2026 – another reason to apply early.

Reassure students that nobody is blocked: Copilot Free works immediately after signing in and is sufficient to follow the first session. Important (GitHub docs "Access GitHub Copilot for free as a student"): after approval, students open their Education benefits, click *Learn more* under the free developer resources and follow the prompts; while the benefit is still being applied, the Copilot settings page is the documented fallback. If only paid options appear, they should not buy anything but wait or ask.

Seven steps – in this order

Later steps depend on earlier ones

	Step	Time
1	GitHub account (with 2FA)	5 min
2	Apply for GitHub Education	10 min
3	Install Git	5–10 min
4	Install Python 3.13	5–10 min
5	Install VS Code + Python ext.	5 min
6	Sign in to Copilot in VS Code	2 min
7	First test with Copilot	10 min

Definition of done

You can run a Python script in VS Code *and* Copilot answers a question in the chat.

Work in pairs

One person installs, the other reads the next step aloud – then swap. Problems are solved twice as fast.

Show the full sequence before starting so students can pace themselves. The times are rough estimates for a laptop with a decent connection; downloads in a full lecture hall can be slower – consider asking students to download the installers beforehand.

The order matters: Git should be installed before VS Code so that VS Code finds it immediately; Python before the first test; the GitHub account before signing in to Copilot.

Walk around the room during this part. The most frequent problems are listed on the troubleshooting slide at the end of this part – keep it open on a second screen.

STEP 1 Create your GitHub account

1. Go to `github.com/signup`
2. Register with your **HSBI email address** (you can add a private address later)
3. Choose a **professional username** – it will appear in your CV and portfolio
4. Confirm your email address
5. Enable **two-factor authentication** (Settings → Password and authentication)

Good usernames

`cweisser`, `christoph-weisser`
not: `xX_coder99_Xx`

Why 2FA now?

GitHub Education requires it – and it protects your code and your licence.

Your GitHub profile is part of your professional identity – treat it like LinkedIn.

STEP 1 Create your GitHub account

Most students will need about five minutes. Encourage a neutral, professional username: recruiters in data and IT roles do look at GitHub profiles, and the POCs of this course are a nice first portfolio.

Two-factor authentication: GitHub's student-benefit application requires 2FA to be enabled (as of Sep 2026). An authenticator app (e.g. Microsoft Authenticator, Google Authenticator) is the easiest option; students should also store the recovery codes safely.

If a student already has a GitHub account with a private email address, they can simply add the HSBI address under Settings → Emails – no second account needed.

STEP 2 Apply for GitHub Education

Verified students get the **GitHub Copilot Student** plan

How to apply

1. Open `github.com/settings/education/benefits`
2. *Start an application*, select HSBI
3. Upload **proof of enrolment** (dated)
4. Allow **location + camera**; **no VPN**

What you get (as of Sep 2026)

unlimited code completions • monthly allowance of AI credits for chat and agent • automatic model selection

Common rejections

blurry or undated document • name differs from your GitHub billing info • VPN switched on

Until approval arrives, simply use Copilot Free.

As of Sep 2026 (plans change – check the docs): since March 2026 verified students get **GitHub Copilot Student** instead of Copilot Pro – agent mode, 200 AI credits per month (from 1 June 2026, replacing "premium requests") and automatic model choice ("Auto").

Requirements in GitHub's docs: currently enrolled, dated proof of enrolment and, if your school's applicants usually verify that way, an academic email address. The verification flow additionally checks location and camera access, no VPN, two-factor authentication, and that the full name in your GitHub billing information matches the document.

Credits are token-based and reset on the 1st of each month without carry-over; agent sessions use them fastest. When they run out, chat and agent pause until the next month (unless someone sets a paid budget – don't); code completions stay unlimited. Usage is shown via the Copilot icon in the Status Bar. Sources: docs.github.com (Copilot plans; usage-based billing for individuals), GitHub's Community announcement on Copilot for Students (the 200 credits) and the GitHub changelog of 13 March 2026.

STEP 3 Install Git

The version-control program on your laptop

Windows

Installer from git-scm.com (default options), or in a terminal:

```
winget install --id Git.Git -e --source winget
```

macOS

Apple's Command Line Tools include Git:

```
xcode-select --install
```

Then, in a new terminal (PowerShell / Terminal): check and introduce yourself

```
git --version                                  # e.g. git version 2.5x
git config --global user.name  "Firstname Lastname"
git config --global user.email "you@hsbi.de"    # same address as on GitHub
```


Git must be installed separately – VS Code has Git support built in, but it needs the Git program itself. On Windows, both the official installer from git-scm.com and winget work; the installer's default options are fine. On macOS, `xcode-select --install` installs Apple's Command Line Tools, including Git; users of Homebrew can alternatively run `brew install git`.

The two `git config` lines tell Git who you are; every commit is labelled with this name and email. Use the same email address as on GitHub so that commits are linked to the profile.

Many students have never opened a terminal: on Windows search for *PowerShell* in the Start menu, on macOS open *Terminal* via Spotlight; from Step 7 on we use the terminal built into VS Code. Tip: close and reopen the terminal (and VS Code) after installing, otherwise the `git` command may not be found yet.

STEP 4 Install Python 3.13

Not the newest version – the one all our packages support

Windows

From python.org/downloads: the *Python install manager*, then:

```
py install 3.13
```

Classic installer: tick "Add python.exe to PATH".

macOS

Download the macOS installer for **Python 3.13.x** from python.org/downloads and run it.

On macOS the command is usually `python3`.

Check in a new terminal

```
python --version    # macOS: python3 --version
Python 3.13.x
```

Why not 3.14 or 3.15?

Brand-new versions such as 3.15 often lack ready-made packages (e.g. PyTorch). Python 3.13 works with everything we use.

As of early October 2026 the newest stable release is Python 3.14 (3.14.8, 30 September 2026); 3.15.0 has slipped to 9 October 2026. We deliberately use 3.13: it is still supported (security fixes until October 2029; 3.13.16 of 30 September 2026 is the last 3.13 release with Windows/macOS installers), and all our packages – including xgboost, numpy, faiss-cpu, chromadb and PyTorch (needed by sentence-transformers in the LLM deck) – provide ready-made wheels for it. For 3.15, PyTorch and onnxruntime had no wheels yet.

Windows: python.org recommends the new *Python install manager* (`py install`, `python`); the classic installer has been deprecated since 3.14 but is still available – if used, tick "Add python.exe to PATH".

Careful on Windows: with the install manager, plain `python` starts the *newest* installed version (and installs the latest one if none exists – that will be 3.15 once it is released). So create environments explicitly with `py -V:3.13 -m venv .venv` (Python docs, "Using Python on Windows"). Check with `py list`. Inside an activated venv, `python` is always the right version.

STEP 5 Install VS Code and the Python extension

1. Download from `code.visualstudio.com` and install with the default options
2. Open the **Extensions** view (`Ctrl+Shift+X` / `Cmd+Shift+X`)
3. Install **Python** by Microsoft (brings Pylance and the debugger)
4. Optional: **Jupyter** by Microsoft

Good news

GitHub Copilot is **built into VS Code**: there is no separate Copilot extension to install any more.

Windows

Keep the option "Add to PATH" ticked – then you can open any folder with `code .`

VS Code is your cockpit: files on the left, code in the middle, terminal below, chat on the right.

VS Code is free and available for Windows, macOS and Linux. The Python extension from Microsoft adds syntax highlighting, code completion via Pylance, running and debugging, and the selection of virtual environments.

Copilot: since early 2026 the former "GitHub Copilot" extension has been deprecated, and all AI features run through the built-in, open-source Copilot Chat integration of VS Code. Students therefore only need to sign in (next step). Older tutorials that tell you to install two Copilot extensions are outdated.

VS Code now ships updates weekly (as of Sep 2026), so menus and labels change often – if a student's screen looks slightly different from yours, that is normal.

STEP 6 Sign in to GitHub Copilot

1. Hover over the **Copilot icon** in the Status Bar (bottom right)
2. Choose **Use AI Features**
3. Sign in with your **GitHub account** in the browser and allow VS Code
4. Back in VS Code: open the chat with `Ctrl+Alt+I` / `Ctrl+Cmd+I`

Check your plan

On github.com: profile picture → *Copilot settings* shows whether you are on *Student* or *Free*.

Privacy

Don't paste personal or confidential company data into the chat.

No subscription yet? Signing in enrols you in Copilot Free automatically.

STEP 6 Sign in to GitHub Copilot

The sign-in path as of September 2026 (VS Code docs "Set up GitHub Copilot"): hover over the Copilot icon in the Status Bar, choose "Use AI Features", pick the sign-in method and authenticate in the browser. Alternatives: the Accounts menu or the command "GitHub Copilot: Sign in". Accounts without a paid or student plan are signed up for Copilot Free.

Mention data protection: prompts and code snippets are sent to GitHub's service for processing. For our synthetic demo data this is unproblematic; for real customer or employee data it would require a company agreement. VS Code also states that telemetry is enabled in the free version and can be switched off in the settings.

If the Copilot icon is missing, check that AI features have not been disabled in the settings.

STEP 7 First test: environment, script, Copilot

Create a folder `hello-poc`, open it via *File → Open Folder*, then *Terminal → New Terminal*:

```
python3.13 -m venv .venv        # macOS
source .venv/bin/activate       # macOS
py -V:3.13 -m venv .venv        # Windows
.venv\Scripts\Activate.ps1      # Windows
```

Create `hello.py` and run it

```
print("Hello -- my setup works!")
```

```
python hello.py
```

Now try Copilot

Inline: type

```
# function that returns the VAT for a net price
```

and press `Enter` – accept the grey suggestion with `Tab`.

Chat (Ask): "Explain `hello.py` and what a virtual environment is."

VS Code asks which Python to use? Pick the one inside `.venv`.

STEP 7 First test: environment, script, Copilot

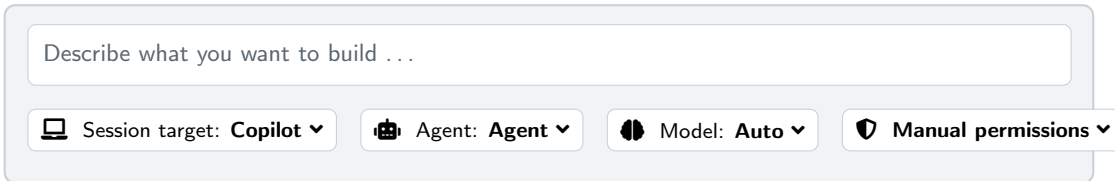
This is the definition-of-done test. Walk through it slowly on the projector. The virtual environment (`.venv`) is a project-specific Python installation – Part 3 explains the concept; for now it is enough that each project gets its own.

Windows PowerShell may refuse to run the activation script ("running scripts is disabled"). Fix, once per user: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`. Alternatively use `.venv\Scripts\activate.bat` in cmd. We avoid `&&` chains because Windows PowerShell 5.1 does not support them, and we open folders via the menu because the `code` command needs extra setup on macOS.

Let students try both: an inline suggestion (generation 1) and a chat question in Ask (generation 2). Ask is offered in the Local session target; in the Copilot target students write "Only explain – do not change any files" in front of the question (see the next slides). Say where to create `hello-poc` (e.g. in Documents) – it is the first question in every room. Congratulate everyone who gets here – the rest of the course builds on it.

The chat view at a glance

Four controls below the input box decide how the agent works



Where it runs.
Start with *Copilot*.

Which mode.
Agent or *Plan* (`/plan`).

Which LLM.
Auto for students.

What needs your OK.
Always *Manual* in class.

Beginner settings: Copilot • Agent (or Plan) • Auto • Manual permissions.

This is a simplified mock-up of the chat input as of September 2026 (VS Code docs "Agent harnesses" and "Approvals"). The four pickers: **Session target** – which harness runs the session and where its tools operate; the docs recommend starting with *Copilot*. **Agent** – the mode with its instructions and tools: in Copilot sessions the default mode (the docs call it *Agent* or *Interactive*), *Plan* (`/plan` switches to it) and *Autopilot* (not in class) – Copilot sessions offer no Ask; only the Local target lists Ask, Plan and Agent (plus custom agents). **Model** – which LLM; on the Free and Student plans only *Auto* is available. **Permissions** – which actions need your confirmation; start with *Manual permissions*.

Because VS Code updates weekly, labels may differ slightly on the students' screens; the concepts stay the same. Ask students to set exactly these four values now and to check them at the start of every session.

Ask, Plan or Agent?

Choose the mode that matches the job

Mode	What it does (VS Code docs)	Use it for ...
ASK	"asks questions and provides guidance without making changes to the code"	understanding code and errors (in the *Local* target)
PLAN	"researches a task and creates a structured implementation plan before code changes"	bigger steps: review the plan first
AGENT	"autonomously plans and performs complex coding tasks, edits files, runs commands, and iterates on results"	building and changing the POCs

Ask for questions, *Plan* before big changes, *Agent* for tasks.
After the plan: *Implement Plan*.

Source: VS Code documentation (Sep 2026)

Ask, Plan or Agent?

The quotes are the official descriptions from the VS Code documentation (Sep 2026, section on Local sessions). In the recommended *Copilot* session target, you switch to planning with the mode selector or `/plan`; the plan appears as a "Review Plan" card with *Implement Plan*, *Approve Plan Only* and *Implement with Autopilot* (never use the latter in class). In Local sessions the button is called *Start Implementation*. If Ask is not offered in your session, simply write "Only explain – do not change any files." The former "Edit" mode was deprecated in March 2026 and removed in June 2026.

Recommended habit for students: when you do not understand something, switch to Ask – it cannot change your files, so exploring is safe. For every POC step that touches several files, start in Plan: read the proposed plan, correct it in plain language, then let the Agent implement it.

Question for the audience: "Which mode would you use to find out why your app crashes?" (Ask – or Agent if you want it to fix the bug, but then review the fix.)

Stay in control: approvals, checkpoints, commits

Approvals

With *Manual permissions* the agent asks before risky actions. Read each command before you click *Allow*.

Checkpoints

Restore Checkpoint resets files and chat to an earlier request – but does **not** undo terminal commands.

Git commits

Your real safety net: commit after every working step, and you can always go back.

Not in this course: "Allow all" and "Autopilot"

They skip all confirmations – including file edits, terminal commands and external tools.

The agent is fast; you are responsible. Approve consciously, commit often.

Source: VS Code documentation: Approvals; Review code edits (Sep 2026)

Facts as of September 2026 (VS Code docs): there are three permission levels – Manual (default), Assisted (experimental: an LLM judge assesses tool calls) and Allow all. Common read-only terminal commands are auto-approved; risky ones such as `rm` or `del` require approval. The docs warn that "Allow all" and "Autopilot" skip confirmation for potentially destructive actions, and that terminal auto-approval is "a best-effort convenience, not a security boundary".

Restore Checkpoint resets the files and chat history to an earlier request, but it cannot reverse terminal commands or changes to external services. In Copilot sessions, edits are saved directly – so students must look at the diff. That is why Git commits are the dependable safety net.

Tell students: if the agent wants to delete files or install something unexpected, click Skip and ask why. In the Agents window, *New Worktree* sessions always run with Allow all – in class, choose *Folder*.

Troubleshooting: the top six

Symptom	Fix
`python/git` not found	reopen terminal + VS Code; Windows: PATH option; macOS: `python3`
VS Code uses the wrong Python	`Ctrl+Shift+P` → *Python: Select Interpreter* → choose `.venv`
PowerShell: "running scripts is disabled"	`Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`
Copilot says "no access"	sign out and in again; until approval, Copilot Free is active
Education application rejected	clear, dated document; billing name as on it; no VPN; location allowed
`pip install` fails	is `(.venv)` shown? Windows: create it with `py -V:3.13`

Read the error message aloud – or paste it into Copilot Ask. Most fixes take a minute.

Keep this slide on a second screen while students install. In our experience the PATH problem on Windows and the interpreter selection in VS Code cause most of the questions.

The PowerShell execution-policy fix is the one recommended in the Python documentation for venv activation on Windows; it only affects the current user.

For the education application, GitHub's docs ("Solving problems with your GitHub Education access") name unclear or undated documents and email-domain issues; GitHub's Community FAQ adds VPN, location/camera access, 2FA and a matching billing name.

Teach the meta-skill: an error message is information, not a failure. Pasting it into Copilot Ask with the question "what does this mean and how do I fix it?" solves most setup problems.

Setup done?

Tick every box before Session 2

- ☑ GitHub account with 2FA
- ☑ Education application submitted
- ☑ `git --version` works, name set
- ☑ `python --version`: 3.13.x
- ☑ VS Code + Python extension
- ☑ Signed in to Copilot
- ☑ `hello.py` runs in a `.venv`
- ☑ Copilot answered in Ask

Homework for Session 2

Ask Copilot (Ask) to explain three terms, then put them in your own words: *repository*, *virtual environment*, *terminal*. Write down one question you still have.

Congratulations – you now have the same toolchain as professional developers.

Setup done?

Close Session 1 with this checklist. Ask students to raise their hands for each item, or run a quick poll. Everybody who is not done should finish at home and post remaining problems in the course forum before Session 2 – the next session assumes a working setup.

The homework does two things: it makes students use Ask mode as a personal tutor, and it prepares the vocabulary for Part 3. Collect the open questions at the start of Session 2.

If time allows, repeat the expense-claim demo in the students' own VS Code: they now have everything they need.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 3 • Session 2 and following: Working with a coding agent

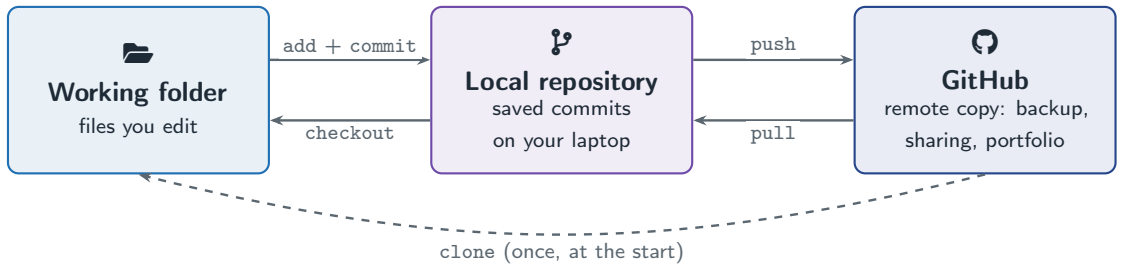
The habits that make AI-assisted coding reliable.

- ✔ Use Git and GitHub as your safety net and portfolio
- ✔ Keep projects clean: environments, `.gitignore`, no secrets
- ✔ Write prompts that produce *your* code, not generic code
- ✔ Review AI-generated code in three stages

Outline

Git and GitHub in five minutes

A time machine for your project – locally and in the cloud



repository	project folder with its full history	**commit**	a saved snapshot with a message
push / **pull**	upload to / download from GitHub	**clone**	copy a GitHub repo to your laptop

Commit = save point. Push = backup. Both cost seconds and save hours.

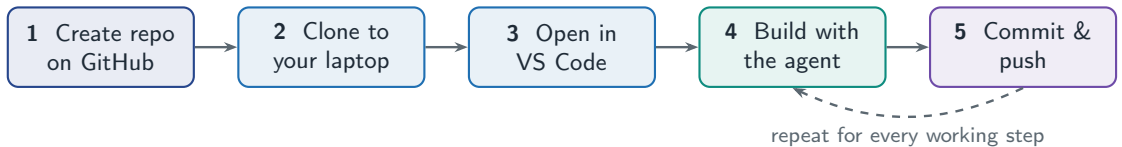
Explain Git with the video-game analogy: a commit is a save point you can always return to. The local repository lives in a hidden `.git` folder inside your project; GitHub holds a remote copy for backup, collaboration and your portfolio.

The four verbs students need in this course: `clone` (once), `add + commit` (often), `push` (after each commit or at the end of a session), `pull` (when working on a second computer or in a team). The `checkout` arrow is the way back: it restores the files of an earlier commit – students rarely need it, but it is what makes a commit a save point. VS Code offers all of them in the Source Control view (`Ctrl+Shift+G`), so students do not have to memorise commands – but they should understand what the buttons do.

Question: "Why is a commit more useful than Copilot's checkpoint?" – it also survives terminal commands and a crashed laptop (after push).

Our workflow: GitHub first

Create the repository online, then work locally



Why this order?

- README, `.gitignore` (template "Python") and licence are created on GitHub with one click
- the connection to GitHub exists from the start – no `git remote add` later
- issues, branches and collaboration work from minute one

Our workflow: GitHub first

This order is the most robust one for beginners. On github.com: New repository, name it (e.g. `ai-automation-pocs`), tick "Add a README", choose the `.gitignore` template "Python", optionally a licence. Then clone it – either with `git clone` in the terminal or in VS Code with the command "Git: Clone" / the Clone Repository button in the Source Control view.

Private or public? For coursework, private is fine; students who want a portfolio can make it public later – but then secrets must never have been committed (next slides).

Alternative for students who already have a local folder: VS Code's "Publish to GitHub" button creates the remote repository afterwards.

Git in practice

Terminal commands – or the same actions as buttons in VS Code

```
# 1. copy the repository to your laptop (once)
git clone https://github.com/<user>/<repo>.git
cd <repo>

# 2. ... let the agent build, review, test ...

# 3. save a working step and upload it
git status                        # what changed?
git add .
git commit -m "POC 1: CSV upload and statistics"
git push
```

Golden rule

Commit after *every* change that works.

Good commit messages

say *what* works now:
"POC 2: /api/upload stores metadata"
not: "changes", "asdf"

Commit small and often – every commit is a save point you can go back to.

Git in practice

Show both ways once: the terminal commands and the Source Control view in VS Code (stage with +, type the message, Commit, then Sync/Push). Copilot can also suggest a commit message from the diff (sparkle icon in the message box) – students should still read it.

Emphasise `git status` before `git add .`: it shows what would be committed. This is where students notice a `.env` file or a huge database that should not be committed.

Tip for the POCs: commit after each item of the run-and-verify checklist. Small commits make it easy to find the step that broke something.

One repository, three POCs

Each POC is self-contained – with its own environment and requirements

```
ai-automation-pocs/
|-- .gitignore      # what Git ignores
|-- README.md       # overview + start commands
|-- poc1_streamlit/
|   |-- .venv/      # local only (ignored)
|   |-- app.py
|   |-- sample_data.py
|   +-- requirements.txt
|-- poc2_fastapi_sqlite/
|   |-- backend/    # FastAPI + SQLite
|   |-- frontend/   # Streamlit
|   +-- requirements.txt
+-- poc3_xgboost_churn/   # like POC 2 + model
```

Conventions

- one repo, one folder per POC
- own `.venv` + `requirements.txt` per POC
- root `README.md` with start commands

What belongs in the repo?

Everything needed to rebuild the project elsewhere – nothing generated, nothing secret.

One repository keeps everything in one place: easy to clone, one history, one README. Each POC gets its own folder, virtual environment and requirements file, so that packages of POC 3 (xgboost, scikit-learn) cannot break POC 1.

The rule in the warning box is the most important one: the repository contains what is needed to reproduce the project on another machine – source code, the list of dependencies, configuration templates and documentation. It does not contain what can be regenerated (virtual environments, databases, caches – and large trained models) or what is secret (API keys, passwords, personal data).

Ask: "Where does the SQLite database of POC 2 belong?" – not in Git; it is created at start-up.

.gitignore: what stays out of the repository

A text file in the repo root that tells Git which files and folders to **ignore**. Without it, repositories become bloated, slow – and unsafe.

Entry	Why
`.venv/`, `__pycache__/`	local, machine-specific
`*.db`, `data/uploads/`	generated at runtime
`.env`, `*.key`	secrets – once pushed = leaked

Example `.gitignore`

```
.venv/
__pycache__/
*.db
data/uploads/
.env
*.key
.streamlit/secrets.toml
```

Check `.gitignore` *before* the first commit – ignoring a file later does not remove it from history.

GitHub's "Python" template (selected when creating the repository) already covers `.venv`, `__pycache__`, `.env` and Streamlit's `secrets.toml`. Trained model files can be committed in a POC if they are small; large ones belong elsewhere.

Important subtlety from the Git documentation: `.gitignore` only affects untracked files. If a file was committed before it was added to `.gitignore`, Git keeps tracking it – and it remains in the history. That is why the check must happen before the first commit.

Note: GitHub's Python template ignores Django's `db.sqlite3`, but not `*.db` or `app.db` – add them. Nice detail: since Python 3.13, `python -m venv` automatically places a `.gitignore` inside the new `.venv` folder.

Secrets and security

API keys and passwords never go into code or Git

Never in the repository

API keys, tokens, passwords, `.env` files with real values, databases with personal data

Instead

Keep secrets in a local `.env` (ignored). Commit a `.env.example` with empty placeholders.

```
# .env.example
LLM_API_KEY=
```

If it happened anyway

1. **revoke or rotate the key at once** (course key: tell the lecturer) – deleting the file is not enough
2. then clean the history (`git filter-repo`) or ask for help

GitHub helps a little

Secret scanning and push protection catch many known key formats in public repos – but not all.

The POCs of this deck need no secrets, but the prototypes in the LLM deck do – for example the course API key that students receive by email. Establish the habit now.

GitHub's documentation (as of Sep 2026) is explicit: if a secret was committed, "as a first step you need to revoke and/or rotate that secret" – that may even be sufficient. Students cannot rotate the course API key themselves: if it leaks, they tell the lecturer at once so that it can be replaced for everyone. Rewriting history is the second step; GitHub now recommends `git-filter-repo` (a separate install, version 2.47 or later) with the `--sensitive-data-removal` option. Bots scan public GitHub repositories for keys within minutes, so rotating is urgent.

Secret scanning runs automatically on public repositories, and push protection for users is on by default – it blocks pushes containing recognised secrets to public repos. It does not cover every format and typically not private student repositories, so it is a safety net, not a strategy.

Virtual environments

Every project gets its own, isolated set of packages

Why?

Projects need different packages and versions. A `.venv` keeps them apart and makes `requirements.txt` reproducible.

In VS Code

`Ctrl+Shift+P` →
Python: Create Environment
or *Python: Select Interpreter*

```
# create (once per project)
python3.13 -m venv .venv        # macOS / Linux
py -V:3.13 -m venv .venv        # Windows

# activate (every new terminal)
source .venv/bin/activate       # macOS / Linux
.venv\Scripts\Activate.ps1      # Windows

# install: by name, or all from the list
pip install streamlit pandas
pip install -r requirements.txt
```

The terminal line starts with `(.venv)`? Then you are in the right environment.

Virtual environments

A virtual environment is a folder containing its own Python interpreter and packages. Activating it changes which `python` and `pip` the terminal uses – recognisable by the `(.venv)` prefix in the terminal prompt. VS Code's Python extension detects `.venv` folders and activates them in new terminals automatically once selected as interpreter.

`requirements.txt` lists the packages a project needs; `pip install -r requirements.txt` recreates the environment on any machine. We let the agent maintain this file – students should check that it contains what the code imports.

Conda environments also work, but venv is built into Python and is what the agent expects by default. uv is a much faster Rust-based alternative (`uv venv`, `uv pip install`) – optional for curious students.

Anatomy of a good coding prompt

Six building blocks that turn generic code into *your* code

1. **Goal** – what should work, for whom
2. **Stack** – FastAPI + SQLAlchemy + SQLite
3. **Files** – `backend/main.py`, `frontend/app.py`
4. **Interfaces & output** – tables, endpoints, JSON fields, UI elements
5. **Demo data** – `sample_data.py`, sample button
6. **Done-criteria** – "done when ...", packages, ports

Why each block matters

Every block you leave out is a decision the agent makes for you – usually the most generic one.

Done-criteria example

"Done when `streamlit run app.py` starts without errors and every numeric column has a histogram."

Bad prompts produce generic code. Good prompts produce *your* code.

Anatomy of a good coding prompt

This is the central slide of the method. The sixth block, explicit done-criteria, is the one beginners forget most often: telling the agent how success will be checked makes it test its own work and reduces "finished" code that does not run.

Link to management practice: a good prompt is like a good work order or user story – goal, context, deliverables, acceptance criteria. Business students already know this structure.

Exercise (3 minutes): students rewrite the demo prompt from Session 1 and mark each of the six blocks in a different colour. Which block was missing? (Usually explicit done-criteria and constraints.)

From a vague to a specific prompt

Vague prompt – leads to generic code

```
Build me an app that can analyse CSVs and does a bit of ML.
```

Specific prompt – leads to *your* code

Create `backend/` with FastAPI + SQLAlchemy + SQLite. GOAL STACK FILES

Models: `Upload`, `AnalysisRun`, `ColumnStat`. Endpoints: `POST /api/upload`, `GET /api/uploads`, `POST /api/analyze/{id}`. Pydantic schemas for all responses. INTERFACES

Demo data via `sample_data.py`. DEMO DATA

Done when all endpoints work in `/docs` with the sample upload. DONE-CRITERIA

The second prompt is longer – and saves you three rounds of corrections.

Read both prompts aloud. Say explicitly that the technical words (FastAPI, SQLAlchemy, Pydantic, endpoints, `/docs`) are the POC 2 vocabulary of Parts 4 and 6 – today only the *structure* matters. Ask the students to predict what the agent will do with the first one: it has to guess the framework, the file structure, the data model, where the data comes from and when it is finished – and it will guess differently every time.

The second prompt names the stack, the files, the data model, the endpoints, the demo data and a done-criterion. Two agents given this prompt will produce very similar, predictable projects. That is what we want for learning – and for business applications.

Students who have seen web tutorials may expect CORS middleware here. For a Streamlit frontend this is unnecessary – Streamlit calls the API from Python on the server, not from the browser. We explain why in Part 4.

Project instructions: tell the agent your rules once

A Markdown file that Copilot reads with every chat request in this repository

.github/copilot-instructions.md

```
# Rules for this repository
- Python 3.13, one .venv per POC folder.
- Keep code simple and commented for
  business students.
- Ask before adding a new dependency;
  keep requirements.txt up to date.
- Never hard-code secrets; use .env.
- After each change: explain it in
  3-5 bullets and update the README
  run commands.
```

Why?

Conventions you would otherwise repeat in every prompt – stated once, applied always.

Alternatives

`AGENTS.md` in the repo root works too. The chat command `/init` drafts a starter file for you.

Create this file in your repository now – short, concrete house rules the agent reads in every POC.

Project instructions: tell the agent your rules once

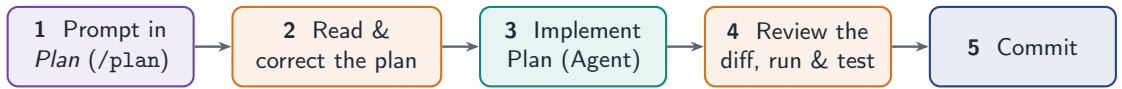
Custom instructions are supported in VS Code for the Copilot harness (docs, Sep 2026): `.github/copilot-instructions.md` for the whole repository, `.github/instructions/*.instructions.md` for rules that apply only to certain files (`applyTo` patterns), and `AGENTS.md`, an open format that several coding agents understand. The chat command `/init` generates a starter file from the existing project.

Have students create the file now – copy the example or run `/init` and trim the result – and commit it; the POC steps assume it exists. Good instruction files are short: coding conventions, environment, what to ask before doing, how to report changes. Long, vague instructions are ignored or cause confusion.

Older tutorials also mention reusable "prompt files" (`.prompt.md`); these are deprecated for Copilot sessions and replaced by "agent skills" – not needed in this course.

Plan first, then build

For every step that touches more than one file



What to look for in a plan

right files? right stack? no surprise dependencies? steps you understand? a way to test?

Correct in plain language

"Use SQLite, not PostgreSQL. Put the API in `backend/`. Skip authentication."

Correcting a plan costs one sentence; correcting finished code costs an afternoon.

The Plan mode researches the task and proposes a structured implementation plan before any file is changed (VS Code docs). In Copilot sessions: `/plan` or the mode selector; the plan card offers *Implement Plan* (in Local sessions: *Start Implementation*). Students review it like a project manager reviews a work plan: are the files and technologies right, is anything unexpected included, can I test the result?

Orange boxes again mark human decisions. Steps 2 and 4 are where students learn most – insist that they read the plan and the diff (VS Code's side-by-side view of old and new lines for every file the agent changed), even if it feels slow at first.

For very small changes ("rename the button", "add a title") going directly to Agent is fine. Rule of thumb: more than one file or a new dependency → plan first.

Three-stage review for AI-generated code

1 Read & understand

Do I understand *every* changed line? If not: ask in Ask – "explain lines 12–18" – *then* accept. Use the diff view.

2 Run & test

Start the app. Try the normal case ("happy path") *and* at least one error case: empty CSV, wrong file type, missing field.

3 Hallucination check

Do the packages exist? Does `pip install` work? Are paths, ports and parameters consistent?

Read it, run it, check it – only then commit it.

This three-stage review is the single most important habit of the course. Stage 1 protects against code you cannot maintain or explain; stage 2 against code that looks right but fails; stage 3 against the specific failure modes of language models.

For stage 2, error cases are key: an empty file, a CSV with text in a numeric column, a missing value. Agents tend to write code for the happy path only. Asking "what happens if the CSV is empty? Add handling and a friendly message" is an excellent follow-up prompt.

Link to the evidence slide: the METR study showed that experienced developers overestimated the benefit of AI. Structured review keeps our judgement honest.

Typical hallucinations in AI-generated code

Plausible-looking, but wrong

Symptoms

- invented packages → `ModuleNotFoundError`
- non-existent functions, outdated APIs
- plausible but wrong defaults or column names
- gaps: `# implementation here`
- "it works" – without having run it

A security angle

Don't install a package just because the AI named it – attackers register look-alike names. Check `pypi.org` first.

Countermeasure

Paste the full error back and ask the agent to fix the *cause*.

If the code sounds too good – install and run it first, *then* believe it.

Why hallucinations happen: the model generates the most plausible continuation, not a verified one (the LLM deck explains why in Parts 1 and 3). Package and function names that "sound right" are especially risky, because they often existed in older versions or in similar libraries.

The security point: if a model repeatedly suggests a package that does not exist, an attacker can register exactly that name on PyPI with malicious code. This supply-chain risk is sometimes called "slopsquatting". Checking the package page (maintainer, downloads, documentation) before installing is a cheap defence.

Good news for the POCs: we use only very popular libraries (pandas, Streamlit, FastAPI, SQLAlchemy, XGBoost), so hallucinated packages are rare – but outdated API usage still happens, e.g. deprecated FastAPI start-up events or old SQLAlchemy syntax.

Top pitfalls – and how to avoid them

	Pitfall	Prevention
1	wrong Python environment	`(.venv)` in the terminal; *Python: Select Interpreter*
2	secrets in the repository	`.env` in `.gitignore` before the first commit
3	"connection refused" in POC 2/3	the backend is not running – start it in a second terminal
4	port already in use (8000 / 8501)	stop the old process (`Ctrl+C`) or use another port
5	huge, unexplained changes	plan first; one feature per prompt; ask for a summary
6	no commits for hours	commit after every working step

Setup before code: most problems appear before the first prompt – or when two processes must talk.

These are the pitfalls beginners hit most often in POC sessions. One item that many online tutorials list is missing on purpose: "CORS errors" do not occur in our architecture. CORS is a browser security rule for JavaScript that calls an API on another origin; Streamlit calls FastAPI from its Python code on the server, so the browser is not involved (MDN; FastAPI docs). CORS becomes relevant only if a JavaScript frontend such as React calls the API directly.

Pitfall 3 is the classic in POC 2: students start only the Streamlit app and get "connection refused" because nothing listens on port 8000. The fix is two terminals, one per process – we practise this in Part 6.

Ask students which pitfall they already hit in Session 1.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 4 • Session 2 and following: How apps are built

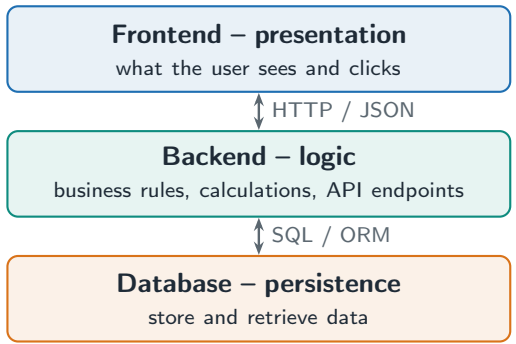
Frontend, backend, database – the vocabulary every automation project uses.

- ✔ Explain the three-tier architecture with an everyday analogy
- ✔ Follow a click from the browser to the database and back
- ✔ Read an HTTP request, a JSON response and a simple SQL query
- ✔ Know what runs where: processes, ports, localhost

Outline

The three-tier architecture

Almost every business application is built in three layers



Restaurant analogy

Dining room = frontend
Kitchen = backend
Pantry = database

The waiter (API) carries orders in a fixed format.

Typical tools:
Streamlit, React
FastAPI, Django
SQLite, PostgreSQL

Architecture answers one question: who calls whom, how – and who stores what.

The three-tier architecture

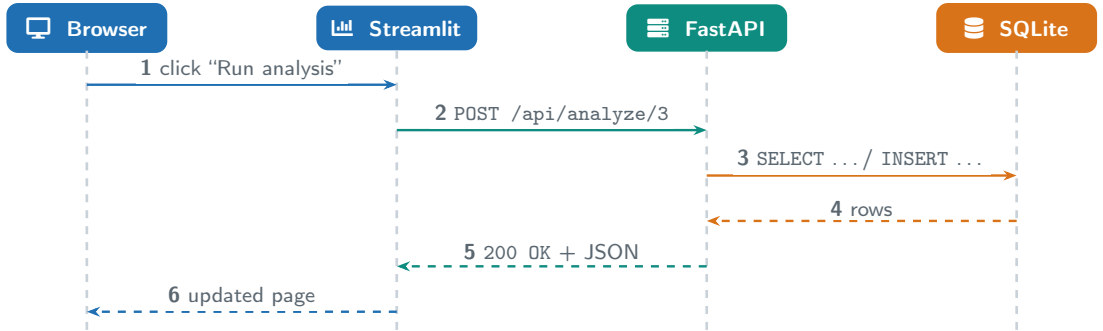
The restaurant analogy works well with business students: guests never walk into the kitchen; they order through a waiter using the menu (a fixed format). The kitchen can change its recipes or staff without redesigning the dining room, and the pantry can be reorganised without the guests noticing.

Translate back: the frontend (Streamlit in our POCs) only shows data and collects input; the backend (FastAPI) contains the business logic and exposes it through an API; the database (SQLite) stores data permanently. Gloss the arrow labels: an endpoint is one address of the API (a path such as `/api/upload` plus GET or POST); an ORM is a library that writes the SQL for us from Python objects – both get their own slide shortly.

Note that the layers are a logical separation. In POC 1 all three live in one Streamlit script; in POC 2 they become separate programs. Seeing the same app in both shapes is the key learning of POC 1 vs. POC 2.

What happens when you click a button?

One request through all three layers (POC 2)



Browser ↔ Streamlit: WebSocket • Streamlit ↔ FastAPI: HTTP • FastAPI ↔ SQLite: SQL via SQLAlchemy

Every layer only talks to its neighbour – that is what makes each one replaceable.

What happens when you click a button?

Walk through the six steps slowly; this picture is the mental model for POC 2 and POC 3. (1) The user clicks a button; the browser tells the Streamlit server via its WebSocket connection. (2) The Streamlit script – running on the server – sends an HTTP request to FastAPI. (3) FastAPI runs the business logic and queries or writes the database through SQLAlchemy. (4) The database returns rows. (5) FastAPI answers with status code 200 and a JSON body. (6) Streamlit reruns the script and sends the updated page to the browser.

Important detail: the browser never talks to FastAPI directly in our setup – that is why CORS plays no role (it is a browser rule). Ask: “Where would you look first if step 2 fails?” (Is the backend running? Is the URL correct?)

HTTP in one slide

The language in which frontends and backends talk

Methods – what you want

GET	retrieve data (no changes)
POST	submit data, create something
PUT	replace an existing resource
DELETE	delete a resource

A request

```
POST http://localhost:8000/api/predict
Content-Type: application/json

{"tenure_months": 3, "monthly_charges": 79.9}
```

Status codes – what happened

200 OK	success
201 Created	new resource created
400 Bad Request	the request is malformed
404 Not Found	wrong path or unknown ID
422 Unprocessable	valid JSON, wrong fields/types
500 Server Error	bug in the backend

2xx = fine
4xx = the caller made a mistake
5xx = the server did

Source: MDN Web Docs (HTTP methods, status codes)

HTTP in one slide

HTTP is the protocol of the web. Every request has a method, a URL (address + path) and optionally a body – in our case JSON. Every response has a status code and usually a JSON body.

The method semantics follow MDN: GET should only retrieve data; POST submits data and often changes state; PUT replaces a resource; DELETE removes it. GET, PUT and DELETE are idempotent (repeating them has the same effect), POST is not – that is why browsers warn before re-submitting a form.

422 ("Unprocessable Content") is the code students will see most in POC 2 and 3: FastAPI returns it automatically when the JSON does not match the Pydantic schema – for example a missing field or text instead of a number – and, in FastAPI, also for malformed JSON. The response body tells you exactly which field is wrong.

An API is a contract

Both sides agree on paths, fields and types – and can then evolve independently

Request (frontend → backend)

```
POST /api/predict
{
  "age": 34,
  "tenure_months": 7,
  "monthly_charges": 74.9,
  "contract_type": "monthly"
}
```

Response (backend → frontend)

```
200 OK
{
  "churn_probability": 0.71,
  "prediction": 1,
  "model_run_id": 3
}
```

FastAPI turns this contract into a live, clickable documentation page at `/docs`.

Agree on the contract first – then build (and prompt) both sides separately.

An API is a contract

An API (application programming interface) is a promise: "if you send me this JSON to this path, I will answer with that JSON." As long as the contract stays the same, the frontend team can redesign the user interface and the backend team can replace the model – neither needs to know the other's code.

This is also why we write two separate prompts in POC 2 (backend, frontend): the endpoint list and the JSON fields are the contract that connects them. If the fields in the two prompts do not match, the app breaks with 422 errors.

JSON in one sentence: text with key–value pairs in curly braces, lists in square brackets; values are strings, numbers, true/false, null or nested objects. The example is POC 3's prediction endpoint with illustrative values; POC 2's endpoints follow the same pattern. An HTTP API built like this – paths, GET/POST, JSON – is what people call a REST API.

Frontend vs. backend – and where Streamlit sits

	Frontend	Backend
Job	show data, collect input	rules, calculations, data access
Talks to	the user	database, models, other systems
Must be	easy to use	correct, secure, reliable
In POC 2/3	Streamlit	FastAPI

Streamlit is special

You write only Python. The script runs **on the server**; the browser just displays the result and sends clicks back (WebSocket).

Consequence

Uploaded data and the app's session values (`st.session_state`) live in the **server's memory** – one session per browser tab.

Streamlit gives you a frontend without HTML or JavaScript – ideal for data prototypes.

In classic web development the frontend is HTML, CSS and JavaScript running in the browser (e.g. React). Streamlit takes a different approach, according to its architecture documentation: the Python script is the server; the browser is only a client connected through a WebSocket. Each browser tab gets its own session, and the script reruns from top to bottom on every interaction.

A common misconception is that the data lives in the "browser session state": in fact `st.session_state` is stored on the server per session – if the Streamlit server restarts, it is gone.

Business consequence: Streamlit is excellent for internal data tools and prototypes; for customer-facing products with many users, companies usually build a JavaScript frontend – and then CORS becomes relevant.

Databases in five minutes

Tables, SQL – and an ORM that writes SQL for you

Table `uploads`

id	filename	n_rows	n_cols
1	sample.csv	500	8
2	sales_q3.csv	1 204	12

id = primary key: unique for every row

SQL: ask the database

```
SELECT filename, n_rows FROM uploads
WHERE n_rows > 1000;
```

SQLite

a complete SQL database in a **single file** (`app.db`) – no server, no configuration, part of Python.

ORM (SQLAlchemy)

maps Python classes to tables: `Upload(filename=...)` becomes a row – the ORM writes the SQL.

Tables store facts, SQL asks questions, the ORM lets Python do both.

A relational database stores data in tables: each row is a record (one upload), each column an attribute. The primary key identifies each row uniquely; other tables refer to it (e.g. `analysis_runs.upload_id`) – that is the "relational" part.

SQL is the standard query language; the example selects all uploads with more than 1,000 rows. Business students often know SQL-like queries from Excel filters or BI tools – make that link.

SQLite describes itself as a "self-contained, serverless, zero-configuration, transactional SQL database engine" stored in a single file, and as the most widely deployed database engine; Python ships the `sqlite3` module. SQLAlchemy is an ORM (object-relational mapper): we define Python classes, and it creates tables and SQL statements. In production, companies swap SQLite for PostgreSQL – with an ORM mainly a change of the connection string, plus a database driver (e.g. psycopg) and a migration tool (e.g. Alembic).

Processes, ports and localhost

Two programs, two terminals, two addresses

Terminal 1 – backend

```
uvicorn backend.main:app --reload
# -> http://127.0.0.1:8000
```

Terminal 2 – frontend

```
streamlit run frontend/app.py
# -> http://localhost:8501
```

process	a running program – it keeps running until you stop it with `Ctrl+C`
localhost (= `127.0.0.1`)	"this computer". FastAPI listens only there; Streamlit also shows a *Network URL*
port	a numbered door of the computer: Streamlit 8501, FastAPI 8000 by default

Both processes must run at the same time – one terminal each.

Processes, ports and localhost

Students often think of an app as one thing. In POC 2 it is two programs: FastAPI served by Uvicorn (default address 127.0.0.1, port 8000) and Streamlit (default port 8501). Each needs its own terminal; closing the terminal or pressing Ctrl+C stops the process.

Localhost means the computer you are sitting at. When the Streamlit script calls `http://localhost:8000`, it means "the FastAPI running on the same machine as Streamlit". If both ran on different servers, this URL would have to change – that is why we put it into a setting (`BACKEND_URL`).

Careful: Uvicorn listens only on 127.0.0.1 by default, but Streamlit listens on all network interfaces and prints a "Network URL" – anyone on the same Wi-Fi (e.g. the lecture hall) can open the app. For local-only use: `streamlit run app.py --server.address localhost`. Typical errors: "address already in use" (an old process still occupies the port – stop it) and "connection refused" (nothing listens on that port – the backend is not running).

Why we separate layers

Benefits and costs

+ Benefits

- **clarity** – each layer has one purpose
- **maintainability** – change the UI without touching the logic
- **reuse** – the same API serves web, mobile, scripts, other systems
- **scalability** – the backend can serve many clients

− Costs

- more moving parts
- communication over HTTP
- two processes to start and watch
- a contract (API) to keep in sync

Start small (POC 1) – add layers only when you need them (POC 2, 3).

Why we separate layers

Architecture is a trade-off, not a virtue in itself. For a one-person data exploration, POC 1 (everything in one Streamlit script) is the better choice: fewer parts, faster to build. As soon as several applications need the same logic, data must be stored permanently, or a model must be served to other systems, the separation pays off.

Automation perspective: an API is what makes a capability usable by other systems – an RPA bot, a workflow tool or an AI agent can call the churn API of POC 3 just as the Streamlit frontend does. This is the bridge to the LLM deck (Part 6): agents use tools, and tools are very often APIs.

Ask: "Which costs would matter most in a company?" (Operations: somebody has to run and monitor two services.)

The role of each technology

One sentence each

Technology	Role in our POCs	POC
Streamlit	frontend in pure Python – ideal for data apps	1, 2, 3
pandas	tables in Python: read CSVs, statistics	1, 2, 3
FastAPI + Pydantic + Uvicorn	backend: REST API with automatic validation and docs	2, 3
SQLite + SQLAlchemy	database in one file, accessed via Python classes	2, 3
XGBoost + scikit-learn	train, evaluate and serve a classification model	3
Git / GitHub	versions, backup, collaboration	all
React, Vue, . . .	professional JavaScript frontends – outlook only	–

Everything is Python – one language from the data to the user interface.

This table is the "cast list" of the POCs. Emphasise that everything is Python, which keeps the cognitive load low for beginners: one language for data processing, backend, database access, machine learning and even the user interface.

Pydantic defines the shape of the JSON (the contract) and validates it; Uvicorn is the server program that runs a FastAPI app; SQLAlchemy is the ORM; scikit-learn provides the train/test split and evaluation metrics around XGBoost.

React and similar frameworks are what companies use for polished, customer-facing web apps. We mention them so that students recognise the terms – replacing Streamlit with React would add HTML, CSS, JavaScript and CORS to the learning load.

How the three POCs build on each other

POC 1 – standalone

- Streamlit app
- CSV upload / sample data
- descriptive statistics
- charts

one process, one file

POC 2 – three tiers

- Streamlit frontend
- FastAPI backend
- SQLite database
- REST API in between

same use case, professional structure

POC 3 – plus ML

- Streamlit + FastAPI
- SQLite database
- XGBoost classifier
- churn prediction + history

data + model + UI = ML app

Each step adds architecture – nothing is taken away.

How the three POCs build on each other

Use this slide as the transition to the hands-on part. POC 1 and POC 2 deliberately solve the same problem, so that the only difference is the architecture – students can compare where data is stored, who calls whom, and what the separation costs. POC 3 keeps the architecture of POC 2 and adds a model layer, which shows that an ML application is "just" a three-tier app with proper model management.

Time planning: Parts 3 and 4 fill about one 90-minute session; POC 1 fits into one session including extensions; POC 2 and POC 3 need about one session each, and the wrap-up (Part 8) adds about 15 minutes at the end. Students who finish early should use the extension challenges rather than jump ahead.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 5 • Session 2 and following: POC 1 – A data app with Streamlit

One file, one process – a useful data tool in under an hour.
✔ Understand Streamlit's rerun model and session state
✔ Build the app with one precise agent prompt
✔ Verify it against acceptance criteria and explain the code

Outline

Use case: understand a CSV in 30 seconds

Scenario

You receive a CSV export – sales, HR, logistics. First you want to know:

- how many rows and columns?
- which data types, how many missing values?
- how are the values distributed?

Goal

One Streamlit app: start it, upload a CSV (or load sample data), get a basic profile and charts immediately.

Deliberately without

backend, database – both come in POC 2.

Data profiling starts every analytics or automation project – worth automating.

Use case: understand a CSV in 30 seconds

The use case is intentionally generic, so that every student can test the app with a CSV from their own context (an internship, a Kaggle dataset, an export from a student job). Data profiling – rows, types, missing values, distributions – is what analysts do by hand at the start of every project; automating it is a small but real productivity gain.

The infobox sets expectations: POC 1 has no backend and no database. Everything happens in one Streamlit script. This makes it the ideal first project to learn Streamlit and the AI-assisted coding loop without architectural complexity.

Ask students which CSV they would like to try – it motivates the testing step later.

Streamlit in 60 seconds

A web app is just a Python script

```python
import pandas as pd
import streamlit as st

st.title("CSV Explorer")
file = st.file_uploader("Upload a CSV", type="csv")

if file is not None:
    df = pd.read_csv(file)
    st.metric("Rows", len(df))
    st.dataframe(df.describe())
    col = st.selectbox("Column", df.columns)
    st.bar_chart(df[col].value_counts())
```

Run it: `streamlit run app.py` → opens `http://localhost:8501`

Three ideas to remember

1 **Rerun:** every click reruns the script top to bottom.

2 **Session state:** `st.session_state` keeps values between reruns.

3 **Caching:** `@st.cache_data` avoids reloading data on every rerun.

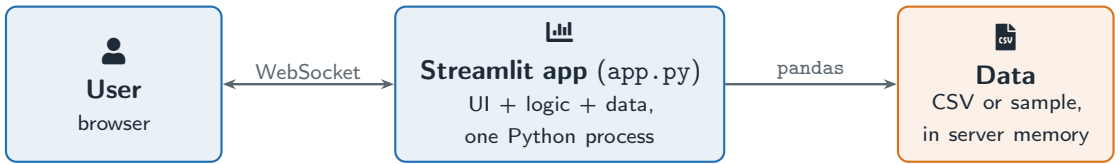
Streamlit in 60 seconds

Show that ten lines of Python produce a complete web app: title, upload widget, a KPI, a table and an interactive chart. Every `st.` function adds an element to the page.

The three ideas are what beginners need to understand the generated code. (1) Streamlit reruns the whole script from top to bottom on every interaction (Streamlit docs) – that is why the code reads like a simple script. (2) Normal variables are reset on each rerun; values that must survive, such as an `upload_id` in POC 2, go into `st.session_state`. (3) `st.cache_data` caches results of functions that return data (e.g. loading a CSV); `st.cache_resource` is for shared objects such as ML models or database connections.

The snippet has been tested with Streamlit 1.64 (Sep 2026).

POC 1 architecture: everything in one process



One process, one language, one file

no separate backend • no database
ideal to learn Streamlit
and the coding loop

Consequence

Data lives only in the Streamlit server's memory, per session: close the tab or restart – and it is gone.

POC 1 architecture: everything in one process

In POC 1, the three logical layers of the architecture all live in `app.py`: presentation (Streamlit widgets), logic (pandas statistics) and data (the DataFrame in memory). There is no persistence: uploads and results exist only for the duration of the browser session on the Streamlit server.

This limitation is the motivation for POC 2: what if we want to keep the history of all uploads and analyses, or reuse the analysis logic from another application? Then we need a database and a backend.

Transition: the next slide shows how to build it, step by step, in agent mode.

POC 1: steps in agent mode

1. Open your **repository root** in VS Code; create folder `poc1_streamlit/`
2. Terminal: `cd poc1_streamlit`, create + activate `.venv`, select it as interpreter
3. Chat: *Agent*, *Manual permissions*; paste the prompt (next slide)
4. Approve the agent's `pip install` and `streamlit run app.py`
5. Run the acceptance tests; iterate; **commit & push**

Timing

prompt + generation: 5 min
review + tests: 15 min
understanding + extensions: rest of the session

Tip

Keep the chat open and the app visible side by side – you will switch often.

POC 1: steps in agent mode

Go through the steps once on the projector, then let students work. Anyone without a cloned repository creates it first (Part 3, "GitHub first"). A one-prompt POC like this does not need Plan mode; for POC 2 and 3 we will plan first.

Why the repository root? Then Copilot reads the repository's `.github/copilot-instructions.md` (instruction files in parent folders are ignored by default), and later prompts can refer to other POC folders. Each prompt therefore names its POC folder ("Work only inside ..."). Remind students to check that the terminal shows `(.venv)` before approving `pip install` – otherwise packages end up in the global Python installation.

Most of the learning happens after generation: in the review, the acceptance tests and the "understand your code" questions. Students who are done in 20 minutes are not finished – they should move on to the questions and the extensions.

POC 1: the prompt

Prompt for Copilot – Agent

Work only inside `poc1_streamlit/` and use its existing `.venv`. Create a Streamlit app `app.py` for a quick first look at a CSV file. Data sources: (a) `st.file_uploader` for my own CSV, (b) a button "Load sample data".

Put the sample data in `sample_data.py`: a function `get_sample_df()` that generates about 500 realistic rows with numpy/pandas (mixed numeric and categorical columns, some missing values).

After loading, show: (1) number of rows and columns, (2) data type and missing values per column, (3) `df.describe()`, (4) a histogram and a box plot (matplotlib) for every numeric column.

Create `requirements.txt` (streamlit, pandas, numpy, matplotlib), `.gitignore` and a short README; install the requirements and start the app.

Done when `streamlit run app.py` works with the sample data. Finally, explain what you created in five bullet points.

GOAL STACK FILES DEMO DATA OUTPUT DONE-CRITERIA

POC 1: the prompt

Two elements make this prompt stronger than a typical first attempt: an explicit done-criterion and the request for a five-bullet explanation. The explanation forces the agent to summarise its work – and gives students a starting point for the review.

Walk through the building blocks with the pills: goal (quick first look at a CSV), stack (Streamlit, numpy/pandas, matplotlib), files (`app.py`, `sample_data.py`, `requirements.txt`), demo data (so the app works without an upload), output (four concrete elements), done-criterion.

Listing the packages in the prompt prevents the agent from adding unnecessary or hallucinated dependencies. If students prefer Streamlit's built-in charts over matplotlib, they can change the prompt – but then the result differs from the reference solution.

POC 1: run and verify

Acceptance tests – tick them before you commit

Happy path

- ☑ app starts without errors
- ☑ "Load sample data" shows about 500 rows
- ☑ missing values per column are plausible
- ☑ every numeric column has a histogram and a box plot

Error cases & checks

- ☑ upload an empty CSV
- ☑ upload a file with only text columns
- ☑ upload a semicolon-separated CSV (Excel export)
- ☑ `requirements.txt` matches the imports

Found a problem?

Describe it precisely: "When I upload an empty CSV, the app crashes with *EmptyDataError*. Catch it and show a friendly message."

A POC is done when it passes your tests – not when the agent says it is done.

POC 1: run and verify

The acceptance tests turn "it looks fine" into a checklist. The happy-path tests check the requirements from the prompt; the error cases check robustness. Agents typically handle the happy path well and forget edge cases – an empty file raises `EmptyDataError` in pandas, a text-only file produces no numeric columns, and a German Excel export is semicolon-separated, so pandas reads one wide column (ask the agent to detect the separator).

Show how to report a bug to the agent: symptom, exact error message, expected behaviour. Vague feedback ("it doesn't work") produces vague fixes.

Commit after the happy path works, and again after each fixed error case. This is the loop from Part 1 in practice.

POC 1: understand your code

Five questions to ask Copilot in *Ask*

1. "Walk me through `app.py` from top to bottom. What happens when I click the sample button?"
2. "Why does the app need `st.session_state` here – or why not?"
3. "How does `get_sample_df()` create missing values?"
4. "What does `df.describe()` calculate, and for which columns?"
5. "Which line would I change to use 30 histogram bins?"

Self-test

Close the chat and explain the app to your neighbour in two minutes. Can you?

Ask (Local target) cannot change files – explore without risk.

If you can't explain it, you don't own it yet.

POC 1: understand your code

This slide operationalises Willison's criterion from Part 1: code you run should be code you can explain. The five questions follow the structure of the app: control flow, state, data generation, statistics, and a small change.

The self-test with the neighbour is surprisingly effective and costs two minutes. Students who struggle should go back to question 1 and ask Copilot to explain it "for a business student without programming experience".

Optional: ask two students to present their explanation to the class. Different generated versions of the same app are a good opportunity to discuss that the agent is not deterministic – and why the review step is therefore essential.

POC 1: extensions

For those who finish early – one feature per prompt

EASY Quick wins

- sidebar filter for a categorical column
- correlation heatmap of numeric columns
- cache loading with `@st.cache_data`

HARDER Mini automation

- download a data-quality report (Markdown/CSV)
- automatic warnings: >20 % missing, constant columns, duplicates
- compare two uploaded CSVs

Each extension = one prompt, one review, one test, one commit.

The extensions deepen the loop without changing the architecture. The "mini automation" ideas turn the profiling app into a data-quality checker that flags issues automatically – a natural bridge back to the expense-claim demo and to the automation theme of the module.

Insist on one feature per prompt. Large combined prompts make review harder and increase the chance of breaking something. After each extension: run the acceptance tests again, then commit.

Students who add the caching extension should observe the difference: without caching, every widget interaction reloads and recomputes; with `@st.cache_data`, the loading function runs only when its input changes.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 6 • Session 2 and following: POC 2 – Frontend, backend and database

Same use case, professional architecture: three layers, two processes, one API.

- ✔ Build a REST API with FastAPI and store results in SQLite
- ✔ Connect a Streamlit frontend to the API over HTTP
- ✔ Run, test and debug two processes that talk to each other

Same use case, new architecture

POC 1 and POC 2 solve the same problem – only the structure changes

What stays the same

- use case: CSV → statistics → charts
- Streamlit as user interface
- pandas for the analysis

What changes

- 1 process → **2 processes**
- memory only → **SQLite database**
- function call → **HTTP API**
- one file → `frontend/` + `backend/`
- no history → **all runs retrievable**

Architecture is "who calls whom, how" – the business logic stays the same.

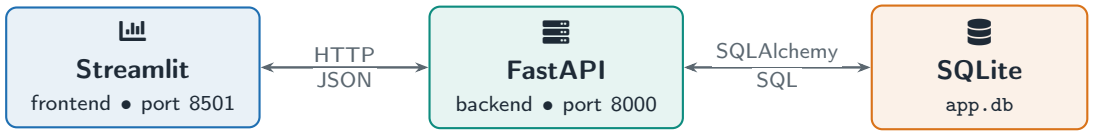
Same use case, new architecture

This comparison is the core insight of POC 2. Because the use case is identical, students can see exactly what architecture adds: a second process, persistence, an HTTP interface and a clear separation of folders – and they can also see what it costs.

Ask before revealing the right column: "What would you need to change so that a colleague can see last month's analysis again?" – persistence. "What if the HR system wants to use the same analysis?" – an API. These are the business reasons for the architecture.

Reassure students: the agent will write most of the extra code, but they must understand the diagram on the next slide to test and debug the result.

POC 2 architecture



Streamlit	UI: data source, upload, buttons, tables, charts – knows only the API URLs
FastAPI	logic: parse CSVs, compute statistics, validate input, read/write the database
SQLite	persistence: upload metadata, analysis runs, column statistics

The frontend never touches the database – everything goes through the API.

POC 2 architecture

Explain the responsibilities with the table. The key rule: the Streamlit frontend only knows HTTP endpoints; it does not import the database code or the statistics functions. That is what makes the backend reusable by other clients and the frontend replaceable (e.g. by a React app later).

SQLAlchemy is the translator between Python objects and SQL; the database itself is a single file, `app.db`, created automatically when the backend starts.

Reminder from Part 4: the HTTP calls go from the Streamlit server process to FastAPI – the browser is not involved, so no CORS configuration is needed.

FastAPI in 60 seconds

An endpoint is a Python function with a URL

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Churn API")
class Customer(BaseModel):          # the contract
    tenure_months: int
    monthly_charges: float

@app.get("/api/health")
def health():
    return {"status": "ok"}

@app.post("/api/predict")
def predict(c: Customer):
    risk = 0.8 if c.tenure_months < 6 else 0.2
    return {"churn_probability": risk}
```

What FastAPI does for you

- turns functions into HTTP endpoints
- validates JSON against the Pydantic model (wrong type → 422)
- generates interactive docs at /docs

```
uvicorn main:app --reload
```


FastAPI in 60 seconds

This minimal example (tested with FastAPI 0.141, Sep 2026) contains everything students need to read the generated backend: a decorator (`@app.get`, `@app.post`) connects a URL and an HTTP method to a Python function; the function's return value becomes the JSON response.

The Pydantic class `Customer` is the contract: FastAPI parses the request body into it and rejects wrong input automatically with status 422 and an explanation of the offending field. The prediction logic here is a placeholder – in POC 3 it will be replaced by the XGBoost model.

Start the server with Uvicorn (default `127.0.0.1:8000`); `--reload` restarts it on every code change. Alternatively, the FastAPI CLI: `fastapi dev main.py`. Then open `http://localhost:8000/docs` and try the endpoints in the browser.

SQLite and SQLAlchemy in 60 seconds

A Python class becomes a database table

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Mapped
from sqlalchemy.orm import mapped_column, Session

class Base(DeclarativeBase):
    pass
class Upload(Base):                # table "uploads"
    __tablename__ = "uploads"
    id: Mapped[int] = mapped_column(primary_key=True)
    filename: Mapped[str]
    n_rows: Mapped[int]

engine = create_engine("sqlite:///./app.db")
Base.metadata.create_all(engine)    # create tables
with Session(engine) as session:
    session.add(Upload(filename="sample.csv", n_rows=500))
    session.commit()
```

Read it like this

class = table
attribute = column
object = row
session = a unit of work: add, commit, query

Want to look inside `app.db`? Install a SQLite viewer extension in VS Code.

This is modern SQLAlchemy 2.x style (DeclarativeBase, Mapped, mapped_column), tested with SQLAlchemy 2.1. Agents sometimes still generate the older 1.x style (`declarative_base()`, `Column(Integer)`) – it still works, but the prompt asks explicitly for 2.0 style for consistency.

The connection string `sqlite:///./app.db` means "a SQLite file called app.db in the folder where the server is started" – a frequent source of confusion when students start the backend from a different folder and suddenly see an empty database.

FastAPI's own SQL tutorial uses SQLModel, a thin layer on top of SQLAlchemy and Pydantic; both approaches are fine. Older tutorials add `check_same_thread=False` for SQLite – with SQLAlchemy 2.x this is set automatically for file databases.

POC 2: project structure

```
poc2_fastapi_sqlite/
|-- .venv/                # one venv for both parts
|-- backend/
|   |-- __init__.py       # makes backend a package
|   |-- main.py           # FastAPI app + endpoints
|   |-- models.py         # SQLAlchemy tables
|   |-- schemas.py        # Pydantic models (contract)
|   |-- database.py       # engine + session
|   +-- sample_data.py    # demo dataset
|-- frontend/
|   +-- app.py            # Streamlit, calls the API
|-- data/uploads/         # stored CSV files (ignored)
|-- app.db                # created at start (ignored)
+-- requirements.txt
```

Separation of concerns

Each backend file has one job – easy to read, easy to prompt ("change only `models.py`").

Run from the project folder

```
uvicorn backend.main:app
streamlit run frontend/app.py
```


The structure follows common FastAPI practice: `main.py` contains the app and the endpoints, `models.py` the database tables, `schemas.py` the Pydantic models that define the JSON contract, `database.py` the engine and session handling. Separating models (database) from schemas (API) is a typical source of confusion – explain that the database may store more than the API reveals.

The `__init__.py` file makes `backend` a Python package, so that Uvicorn can be started from the project folder with `backend.main:app`. Both commands run from `poc2_fastapi_sqlite/`; this keeps relative paths such as `./app.db` predictable.

Uploaded CSV files and the database are generated at runtime and belong in `.gitignore`.

POC 2: the API contract

Endpoints the frontend may call

Method	Path	Purpose
POST	`/api/upload`	receive a CSV, store file + metadata, return `upload_id`
POST	`/api/sample`	create (or return) the sample upload – testing without a file
GET	`/api/uploads`	list all uploads with timestamp
POST	`/api/analyze/{id}`	start an analysis run, compute and store statistics
GET	`/api/stats/{id}`	descriptive statistics per column
GET	`/api/plots/{id}`	histogram data per numeric column

Test every endpoint in `http://localhost:8000/docs` before you build the frontend.

The endpoint list is the contract between the two prompts. Note the pattern: POST for actions that create or change something (upload, sample, analyze), GET for reading (list, stats, plots). `{id}` is a path parameter – the ID of the upload.

`/api/sample` is a small but important design decision: it lets you test the whole chain without preparing a CSV file, which saves a lot of time in class.

The plots endpoint returns data (bin edges and counts), not images. The frontend decides how to draw them – a clean separation between logic and presentation.

POC 2: database tables

Table	Columns (example)
`uploads`	`id`, `filename`, `n_rows`, `n_cols`, `uploaded_at`
`analysis_runs`	`id`, `upload_id` (→ `uploads`), `started_at`, `finished_at`, `status`
`column_stats`	`id`, `run_id` (→ `analysis_runs`), `column`, `dtype`, `n_missing`, `mean`, `std`, `min`, `max`

The payoff

Close the app, come back tomorrow – all uploads and results are still there.

Relations

One upload → many runs; one run → one stats row per column.

POC 2: database tables

Three tables with one-to-many relations: each upload can be analysed several times (runs), each run stores one statistics row per column. The arrows mark foreign keys – columns that refer to the primary key of another table.

This is also a small audit trail: you can see when which file was analysed and with which result. In business contexts such traceability is often a requirement (e.g. for controlling or compliance).

Exercise: ask students to open `app.db` with a SQLite viewer after two analyses and find the rows that belong to their last run.

POC 2: steps in agent mode

Plan first – this step touches many files

1. Repo root open; create `poc2_fastapi_sqlite/`; own `.venv`, activated and selected
2. *Plan* (`/plan`): backend prompt, review the plan, *Implement Plan*
3. Start the backend, test all endpoints in `/docs`, **commit**
4. Frontend prompt in *Agent*; start Streamlit in a second terminal
5. Test the full chain, iterate, **commit & push**

Backend first

A backend you have tested in `/docs` makes frontend bugs easy to locate.

Check the plan for

SQLite (not PostgreSQL), no login, no CORS, the files from the structure slide

POC 2: steps in agent mode

The order matters: build and test the backend on its own first. If the frontend later shows an error, students know the backend works – the bug must be in the frontend or in the connection.

In the plan review, the typical deviations are: a different database (PostgreSQL), extra features (authentication, Docker), CORS middleware (harmless, but a sign the agent assumes a JavaScript frontend), or a different file layout. Students correct these in one sentence before implementation starts.

Commit after step 3 with a message like "POC 2: backend with endpoints tested in /docs".

POC 2: prompt 1 – backend

Prompt for Copilot – /plan, then Implement Plan

Work only inside `poc2_fastapi_sqlite/` and use its existing `.venv`. Create a FastAPI backend in `backend/` (a package with `__init__.py`) using SQLAlchemy 2.0 style and SQLite (`app.db`). Files: `database.py`, `models.py` (Upload, AnalysisRun, ColumnStat), `schemas.py` (Pydantic), `sample_data.py` (about 500 realistic demo rows), `main.py`.

Endpoints: POST `/api/upload` (store the CSV in `data/uploads/`, save metadata, return `upload_id`), POST `/api/sample` (create or return the sample upload, return its `upload_id`), GET `/api/uploads`, POST `/api/analyze/{id}`, GET `/api/stats/{id}`, GET `/api/plots/{id}` (histogram data per numeric column).

Use a lifespan function to create the tables and seed the sample upload. No CORS middleware. `requirements.txt`: `fastapi[standard]`, sqlalchemy, pandas – install it. Start command: `uvicorn backend.main:app --reload`.

Done when every endpoint works in `/docs`.

POC 2: prompt 1 – backend

Three details in this prompt prevent typical agent mistakes. (1) SQLAlchemy 2.0 style, so that the generated code matches the snippet students saw. (2) A lifespan function for start-up work: FastAPI's documentation recommends the `lifespan` context manager and marks the old `@app.on_event("startup")` as deprecated. (3) No CORS middleware – it is not needed for a Streamlit frontend.

The prompt contains all six building blocks: goal (backend for the CSV analysis), stack, files, interfaces (tables and endpoints), demo data (sample upload seeded at start) and a done-criterion (all endpoints work in `/docs`).

Paste it into Plan first (`/plan`), then *Implement Plan*. Typical plan corrections: "store the uploaded CSV files on disk, not in the database", "use the exact endpoint paths from the prompt".

Why the backend prompt works

Stack and structure fixed

FastAPI, SQLAlchemy 2.0, SQLite, file names – no guesswork for the agent.

Endpoints defined

Paths and methods are the contract the frontend prompt will rely on.

Data model given

Three tables structure uploads, runs and column statistics.

Testable from minute one

Seeded sample upload + `/docs` = a done-criterion you can check.

A good backend prompt reads like a mini specification – because it is one.

Why the backend prompt works

Use this slide to reflect on prompt quality after students have seen the result. Each card corresponds to a decision that would otherwise be left to the agent.

Ask students to compare their generated backends: do all of them have the same endpoints? The same file names? Where do they differ? Differences usually appear in details the prompt did not specify – for example the exact JSON structure of `/api/stats`. That is a good moment to discuss how much specification is enough for a POC.

Business analogy: this prompt is what a product owner would write as acceptance criteria for a development team.

POC 2: prompt 2 – frontend

Prompt for Copilot – Agent

Create `frontend/app.py` (Streamlit) in `poc2_fastapi_sqlite/` that talks to the backend **only** via HTTP with `requests`; base URL from the environment variable `BACKEND_URL` (default `http://localhost:8000`).

A `st.radio` selects the data source: "My CSV" (`st.file_uploader` + button "Upload" → POST `/api/upload`, each file only once) or "Sample data" (button "Load sample data" → POST `/api/sample`). Store the returned `upload_id` in `st.session_state`.

A button "Run analysis" calls POST `/api/analyze/{id}`; show GET `/api/stats/{id}` as a table and GET `/api/plots/{id}` as histograms. A sidebar lists earlier uploads (GET `/api/uploads`).

If the backend is not reachable, show a clear error message. Add streamlit, requests and matplotlib to `requirements.txt` and install them. Done when "Load sample data" → "Run analysis" shows the table and the histograms.

POC 2: prompt 2 – frontend

The frontend prompt references the endpoints of the backend prompt exactly – that is the contract in action. Three details are worth pointing out. (1) "only via HTTP" prevents the agent from importing backend code directly into the frontend, which would silently undo the architecture. (2) The base URL comes from an environment variable with a sensible default, so the frontend also works when the backend runs elsewhere. (3) `st.session_state` keeps the `upload_id` across Streamlit's reruns – without it, the ID would be lost as soon as the user clicks "Run analysis".

The error message for an unreachable backend is a usability detail that saves a lot of confusion in class ("connection refused"). The explicit "Upload" button matters because `st.file_uploader` returns the same file on every rerun – naive code would re-send it on every click and create duplicate uploads.

Why the frontend prompt works

The frontend knows URLs – nothing else

```python
import os
import requests
import streamlit as st

BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:8000")     # configurable

if st.button("Load sample data"):
    r = requests.post(f"{BACKEND_URL}/api/sample", timeout=10)      # HTTP call
    r.raise_for_status()                                            # 4xx/5xx -> error
    st.session_state["upload_id"] = r.json()["upload_id"]           # survives reruns
```

Separation & state

No import from `backend/` – the frontend could be replaced by a mobile app or a script. `st.session_state` keeps the `upload_id` for the next click.

This is the core pattern of every Streamlit-to-API call: build the URL from a configurable base, send the request with a timeout, raise an error for 4xx/5xx status codes, and read the JSON. Students will find a similar structure in their generated frontend – ask them to locate it.

`raise_for_status()` turns an HTTP error into a Python exception, which Streamlit shows as an error box; a good frontend catches `requests.exceptions.ConnectionError` and shows a friendly message instead (that is what the prompt asked for).

Connection to automation: any other program – an RPA bot, a workflow tool, an AI agent – could call the same endpoints with exactly these few lines. That is the value of an API.

POC 2: run and verify

Two terminals – then the acceptance tests

Terminal 1 – backend

```
cd poc2_fastapi_sqlite
source .venv/bin/activate
uvicorn backend.main:app --reload
```

then open `http://localhost:8000/docs`

Windows: `.venv\Scripts\Activate.ps1`

Terminal 2 – frontend

```
cd poc2_fastapi_sqlite
source .venv/bin/activate
streamlit run frontend/app.py
```

opens `http://localhost:8501`

Windows: `.venv\Scripts\Activate.ps1`

Acceptance tests

- ☑ all six endpoints work in `/docs`
- ☑ sample data → analysis → table + charts
- ☑ restart both – old uploads still listed
- ☑ stop the backend – friendly error in the UI

Demonstrate the two-terminal setup on the projector: VS Code's terminal panel has a "+" button (or split view) for a second terminal. On Windows, replace the activation line with `.venv\Scripts\Activate.ps1`.

The acceptance tests check the three layers: the API on its own (`/docs`), the full chain, persistence (restart and see old uploads) and robustness (stop the backend and look at the error message). The persistence test is the moment students "feel" the difference from POC 1.

Stop both processes with Ctrl+C at the end – otherwise the ports stay busy and the next start fails with "address already in use".

Debugging the three layers

Where to look when something breaks

Symptom	Likely layer	What to do
"connection refused"	backend not running	start Uvicorn in terminal 1; check the port
404 Not Found	wrong path	compare the URL with `/docs`
405 Method Not Allowed	wrong method	GET used on a POST endpoint (or vice versa)
422 Unprocessable Content	contract mismatch	read the error body: which field is wrong?
500 Internal Server Error	backend bug	read the traceback in terminal 1, paste it to the agent
empty list after restart	database path	started from another folder? `./app.db` is relative
"address already in use"	old process	stop it with `Ctrl+C` or use another port

Debug from the bottom up: database → API in `/docs` → frontend.

Debugging the three layers

Debugging a multi-process app is a new skill for most students. The table maps symptoms to layers. The most useful habit: test the layer below first. If an endpoint fails in `/docs`, the frontend is not the problem. If it works in `/docs` but not from Streamlit, compare URL, method and JSON fields.

Terminal 1 (Uvicorn) shows every request with method, path and status code, plus full tracebacks for 500 errors. Students should learn to read these logs – and paste them into the chat when asking the agent for a fix.

The "empty list after restart" symptom comes from the relative database path: starting Uvicorn from a different folder creates a new, empty `app.db` there.

POC 2: extensions

EASY **Quick wins**

- delete an upload (DELETE `/api/uploads/{id}`)
- show the run history per upload
- download the statistics as CSV

HARDER **Towards production**

- automatic data-quality rules stored per upload
- a small Python script that calls the API – no UI
- tests for two endpoints with FastAPI's `TestClient`

The script-without-UI extension shows why APIs matter for automation: machines can use them too.

POC 2: extensions

The extensions practise the API contract: every new feature requires a backend change (endpoint), a frontend change (button/table) and a test. Encourage students to plan such features in Plan mode and to commit after each layer.

The "script that calls the API" extension is particularly instructive for this module: a ten-line Python script uploads a file, runs the analysis and prints the result – no human, no UI. That is exactly how automation workflows and AI agents use existing systems.

Writing two tests with `TestClient` gives a first glimpse of professional software practice without leaving the POC mindset.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 7 • Session 2 and following: POC 3 – Adding a machine-learning model

From descriptive statistics to predictions: data → model → API → UI.

- ✔ Train and evaluate an XGBoost classifier on churn data
- ✔ Serve predictions through the FastAPI backend and log them
- ✔ Judge a model with the right metrics – and use it responsibly

Outline

What changes from POC 2 to POC 3

What stays the same

- three-tier architecture
- REST API with Pydantic schemas
- persistence in SQLite
- Streamlit frontend calling the API

What is added

- a **model layer**: `model.json` + metadata
- data generator and training script
- prediction endpoint returning a probability
- prediction history (audit trail)
- model-run table with metrics

An ML app is a three-tier app plus careful model management.

What changes from POC 2 to POC 3

The recap makes the progression explicit: nothing from POC 2 is thrown away. The new element is a model layer – a trained XGBoost model stored as a file, loaded by the backend, and used by a new endpoint.

Two additions are easy to overlook but essential in business practice: the prediction history (which customer got which score from which model version, and when) and the model-run table (when was the model trained, on how much data, with which metrics). Together they make predictions traceable – a basic requirement for accountable use of ML, and an early form of what the EU AI Act asks for high-risk systems (logging, documentation).

Question: "Why would a sales manager want to know which model version produced a score?"

Use case: predict customer churn

Scenario

A telecom provider wants to know which customers are likely to cancel their contract (*churn*) and contact them proactively. The sales team needs a web app to:

- **browse** existing customer data
- get a **churn probability** per customer
- keep a **log** of all predictions

Why this is a good ML example

- clear **binary** target: churn yes/no
- realistic mix of numeric and categorical features
- direct business value, easy to motivate
- small enough to build end to end in one session

Retention is usually cheaper than acquisition – if you know whom to call.

Churn prediction is a classic business application of machine learning: marketing and sales want to spend retention budgets (discounts, calls, upgrades) on the customers most likely to leave. The model does not decide anything – it prioritises a list for humans.

The scenario is deliberately simple: one row per customer, a handful of features, one binary target. This keeps the focus on the architecture and on evaluation, not on feature engineering.

The statement in the take-away is a widely used rule of thumb in marketing; the exact ratio varies by industry, so present it as a motivation, not as a fact with a number.

The data

One row per customer – synthetic, but realistic

id	age	contract_type	tenure_months	monthly_charges	payment_method	support_calls	churn
1001	34	monthly	7	74.90	credit card	3	1
1002	58	two years	48	99.50	bank transfer	0	0
1003	27	monthly	2	59.95	credit card	4	1

```
features = ["age", "tenure_months", "monthly_charges",
            "contract_type", "payment_method",
            "support_calls"]
target   = "churn"      # 0 = stays, 1 = cancels
```

Why synthetic data?

No personal data, no licence questions – and we control how features relate to churn (about 20 % churn rate). The agent generates it with numpy.

The three rows are illustrative. The agent will generate a synthetic dataset of about 5,000 customers in which short tenure, monthly contracts and many support calls increase the churn probability – so the model has real patterns to learn.

Optional real-data alternative for advanced students: the "Telco Customer Churn" sample dataset published by IBM (7,043 customers, 21 columns, about 26.5% churn; 11 blank values in TotalCharges that need cleaning). Its licence status is not entirely clear (IBM code pattern under Apache-2.0; the Kaggle mirror lists "data files © original authors"), so use it for learning only.

Features are the inputs, the target is what we want to predict. Point out that `id` is not a feature – a classic mistake.

Machine learning in 60 seconds

Learn from labelled examples, test on unseen ones

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
from xgboost import XGBClassifier

df = pd.read_csv("data/customers.csv")
X = pd.get_dummies(df.drop(columns=["id", "churn"]))
y = df["churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

model = XGBClassifier(n_estimators=300, max_depth=4,
                      learning_rate=0.05)
model.fit(X_train, y_train)                  # learn
proba = model.predict_proba(X_test)[:, 1]    # predict
print("ROC-AUC:", roc_auc_score(y_test, proba))
model.save_model("backend/model.json")       # persist
```

The recipe

1. encode categories (one-hot)
2. split: 80 % train, 20 % test
3. fit the model
4. evaluate on the test set
5. save the model

The output is a **probability**, not a verdict.

Machine learning in 60 seconds

This is supervised learning in one slide (code tested with pandas, scikit-learn and xgboost in Sep 2026). `get_dummies` turns categorical columns such as `contract_type` into 0/1 columns (one-hot encoding). The stratified split keeps the churn rate equal in training and test data. ROC-AUC in the print line is explained two slides on – for now: 1.0 is perfect, 0.5 is a coin toss. The model learns on 80% and is evaluated on the 20% it has never seen – otherwise we would measure memorisation, not prediction.

`predict_proba` returns a probability for each class; column 1 is the churn probability. `save_model` writes the model to JSON, the format recommended by the XGBoost documentation for storing models (pickle files may not load in later versions).

Important for serving: the backend must build exactly the same one-hot columns as in training – that is why we also save the column list (`model_meta.json`).

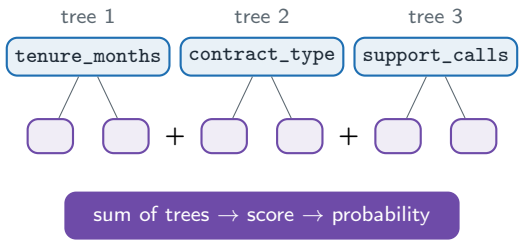
Why XGBoost?

A strong default for tabular business data

Gradient-boosted decision trees

Many small decision trees, trained one after another – each new tree corrects the errors of the previous ones.

- strong default on **tabular data** – often better than neural networks there
- handles **missing values** natively
- **feature importance** for a first explanation
- fast to train on a laptop, a few lines of code



Source: Chen & Guestrin (2016); Grinsztajn et al. (2022)

Why XGBoost?

XGBoost (Chen & Guestrin, KDD 2016) is an efficient implementation of gradient boosting: an ensemble of shallow decision trees, built sequentially so that each tree fits the remaining errors. Its "sparsity-aware split finding" learns a default direction for missing values – no imputation needed.

Grinsztajn, Oyallon and Varoquaux (NeurIPS 2022) showed on a large benchmark that tree-based models still outperform deep learning on typical tabular data – which is exactly what most business data looks like (tables from ERP, CRM, finance).

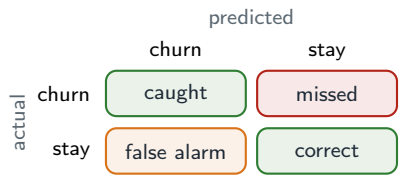
Feature importance (e.g. `model.feature_importances_`) gives a first, rough explanation of what drives predictions; it is not a causal statement. For individual explanations, methods like SHAP are used – an extension for interested students. We do not derive the mathematics in this course.

Is the model any good?

Accuracy alone can fool you

The accuracy trap

With 20 % churn, a "model" that always says *no churn* is right 80 % of the time – and useless.



Metric	Question it answers
Precision	Of the customers we call, how many would really leave?
Recall	Of all real churners, how many do we catch?
ROC-AUC	How well does the score rank churners above stayers? (0.5 = random)
Baseline	What does "always no churn" achieve?

A missed churner costs a customer, a false alarm a discount – the threshold is a business decision.

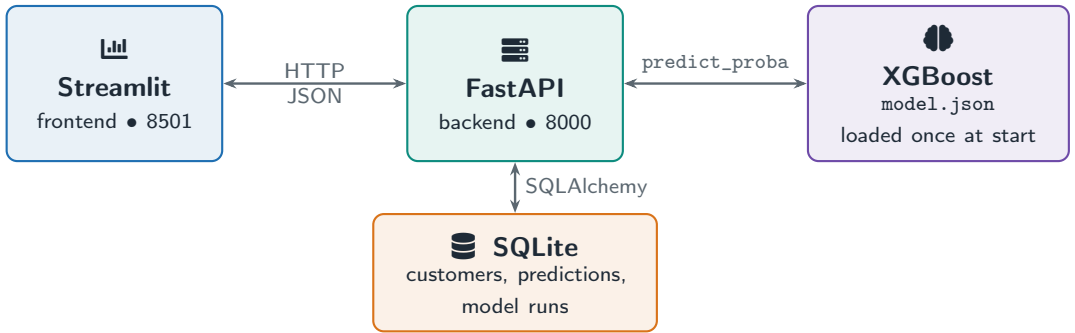
Is the model any good?

This slide is where business students can outperform pure technicians. With imbalanced classes, accuracy is misleading: the majority-class baseline (scikit-learn's `DummyClassifier(strategy="most_frequent")`) already reaches 80% with 20% churn. Always report the baseline next to the model.

Tie the metrics to the matrix: recall = caught ÷ (caught + missed), precision = caught ÷ (caught + false alarms); the threshold is the probability above which we count a customer as "churn" and call them. Precision and recall describe the two kinds of error in the confusion matrix; ROC-AUC measures how well the probability ranks churners above non-churners across all thresholds (1.0 perfect, 0.5 random). The prompt therefore asks the training script to print accuracy, ROC-AUC, precision, recall and the dummy baseline.

Business framing: the threshold (e.g. call everyone above 0.6) is a management decision based on the cost of a missed churner versus the cost of a retention offer – not a technical default of 0.5.

POC 3 architecture



Data flow of one prediction

Streamlit → `POST /api/predict` → FastAPI builds the feature vector → model returns a probability → row in `predictions` → JSON back → Streamlit shows the score.

POC 3 architecture

The backend loads the model once at start-up in the lifespan function – FastAPI's own documentation uses exactly this example (loading an ML model) for lifespan events. Loading the model on every request would be slow and wasteful.

Walk through the data flow: the frontend sends the customer's features as JSON; FastAPI validates them with Pydantic, builds the same one-hot columns as in training (using the stored column list), calls `predict_proba`, stores the prediction with a timestamp and the model-run ID, and returns the probability.

Point out that the frontend still knows nothing about XGBoost. If the data-science team replaces the model tomorrow, the frontend does not change – the API contract stays the same.

POC 3: project structure

```
poc3_xgboost_churn/
|-- .venv/
|-- backend/
|   |-- __init__.py
|   |-- main.py             # FastAPI app + endpoints
|   |-- models.py           # SQLAlchemy tables
|   |-- schemas.py          # Pydantic models
|   |-- database.py         # engine + session
|   |-- generate_data.py    # synthetic churn data
|   |-- train_model.py      # training + evaluation
|   |-- model.json          # trained XGBoost model
|   +-- model_meta.json     # feature columns, metrics
|-- frontend/app.py         # Streamlit UI
|-- data/customers.csv      # generated
|-- app.db                  # generated (ignored)
+-- requirements.txt
```

New files

`generate_data.py`, `train_model.py`, `model.json`, `model_meta.json`

Training–serving skew

The backend must build *exactly* the columns the model was trained on – hence `model_meta.json`.

POC 3: project structure

Compared to POC 2, the backend gets two scripts (data generation and training) and two artefacts (the model and its metadata). Training is a separate step, run once before starting the API – or later via `POST /api/train`.

The model is saved with XGBoost's `save_model` as JSON rather than with pickle/joblib, following the XGBoost documentation's recommendation for long-term storage; the one-hot column list and the metrics go into a small JSON metadata file.

"Training–serving skew" is a classic production bug: if the backend one-hot-encodes a request differently from training, the model receives different inputs. With pandas DataFrames XGBoost usually complains loudly (feature-name mismatch) – the dangerous case is silent: a misspelled category such as "Monthly" becomes all zeros after reindexing. Reindexing to the stored column list plus strict category types (`Literal`) prevents both – the backend prompt asks for it.

POC 3: API endpoints and tables

Method	Path	Purpose
GET	`/api/customers`, `/api/customers/{id}`	list / details of stored customers
POST	`/api/predict`	prediction for a feature vector (JSON)
POST	`/api/predict/{customer_id}`	prediction for a stored customer
GET	`/api/sample-customer`	a random example profile for testing
POST	`/api/train`	retrain the model (optional)
GET	`/api/model/info`, `/api/predictions`	model metrics; prediction history

Table	Columns (example)
`customers`	`id, age, contract_type, tenure_months, monthly_charges, payment_method, support_calls, churn`
`predictions`	`id, customer_id, probability, prediction, model_run_id, created_at`
`model_runs`	`id, trained_at, n_samples, accuracy, roc_auc, precision, recall, model_path`

POC 3: API endpoints and tables

The API extends POC 2's pattern: GET for reading customers, model information and history; POST for predictions and retraining. `/api/sample-customer` plays the same role as `/api/sample` in POC 2 – it makes testing possible without typing a full feature vector.

The three tables make every prediction traceable: which customer, which probability, which model run, when. The model-run table stores the evaluation metrics of each training, so that a retrained model can be compared with the previous one before it is used.

In business terms, this is a minimal "model registry" and "prediction log" – concepts that MLOps platforms offer at scale.

POC 3: steps in agent mode

1. Repo root open; create `poc3_xgboost_churn/`; own `.venv`, activated and selected
2. **Data & model:** prompt 1 – run training, read the metrics, **commit**
3. **Backend:** prompt 2 (`/plan` → Implement Plan) – test in `/docs`, **commit**
4. **Frontend:** prompt 3 – start both processes, test, **commit**
5. Iterate: feature importance, threshold, batch scoring

Checkpoint after step 2

Does the model beat the "always no churn" baseline? If not, fix the data before building an API.

Reuse

Tell the agent: "Use the same structure and conventions as `poc2_fastapi_sqlite`."

POC 3: steps in agent mode

Three prompts, three commits. The order data → model → API → UI is the natural order of every ML application; each stage is tested before the next one is built.

The checkpoint after step 2 is a professional habit: there is no point in building an API for a model that is not better than a trivial rule. Students should look at the printed metrics and compare ROC-AUC and recall with the baseline.

The reuse tip is powerful with coding agents: pointing to an existing folder gives the agent a concrete template for structure and style – the result is more consistent than a description from scratch.

POC 3: prompt 1 – data and model

Prompt for Copilot – Agent

Work only inside `poc3_xgboost_churn/` and use its existing `.venv`. Create `backend/generate_data.py`: generate a realistic synthetic churn dataset with numpy (about 5,000 rows; fields `id`, `age`, `tenure_months`, `monthly_charges`, `contract_type`, `payment_method`, `support_calls`, `churn`; churn rate about 20 %, plausibly related to the features) and save it as `data/customers.csv`. Create `backend/train_model.py`: run the generator if the CSV is missing; one-hot encode the categorical columns with pandas; stratified 80/20 split; train an `XGBClassifier`; print accuracy, ROC-AUC, precision, recall and the baseline of always predicting "no churn". Save the model with `save_model("backend/model.json")` and the feature columns plus metrics to `backend/model_meta.json`. Use package imports (`backend.generate_data`). Create `requirements.txt` (numpy, pandas, scikit-learn, xgboost), install it and run `python -m backend.train_model`. Done when the metrics are printed and `model.json` exists.

POC 3: prompt 1 – data and model

The prompt fixes the data schema, the churn rate and the relationship between features and target, so that the model has something to learn. Asking for "plausibly related" features typically yields rules like: short tenure, monthly contracts and many support calls increase the churn probability.

The evaluation line implements the previous slide: several metrics plus the baseline. Saving the column list and metrics to `model_meta.json` prepares the backend (column alignment) and the `/api/model/info` endpoint.

The package-import instruction makes the same code work both as a script (`python -m backend.train_model`, run from the project folder) and when the backend imports it for `POST /api/train`. After running it, discuss the output with the class: is ROC-AUC clearly above 0.5? Is recall acceptable? If the synthetic relationships are too strong, metrics look unrealistically perfect – a good opportunity to discuss that real data is messier.

POC 3: prompt 2 – backend

Prompt for Copilot – /plan, then Implement Plan

Create the FastAPI backend in `poc3_xgboost_churn/backend/` with the same structure and conventions as `poc2_fastapi_sqlite`. SQLAlchemy 2.0 models: Customer, Prediction, ModelRun. Endpoints: GET `/api/customers`, GET `/api/customers/{id}`, POST `/api/predict`, POST `/api/predict/{customer_id}`, GET `/api/sample-customer`, POST `/api/train`, GET `/api/model/info`, GET `/api/predictions`. In a lifespan function: create the tables, import `data/customers.csv` if the customers table is empty, load `model.json` and `model_meta.json` once, and create a ModelRun from the metadata if none exists. For each prediction, build the one-hot columns exactly as in `model_meta.json`, store the prediction with the model run and return `churn_probability`, `prediction` and `model_run_id`. `POST /api/train` adds a new ModelRun and reloads the model. Use `Literal[...]` for the categories. No CORS middleware. Add `fastapi[standard]` and `sqlalchemy` to `requirements.txt` and install them. Done when every endpoint works in `/docs`.

POC 3: prompt 2 – backend

Key elements of this prompt: (1) reuse of the POC 2 conventions, (2) the complete endpoint list as the contract, (3) start-up work in the lifespan function – creating tables, seeding customers and loading the model once, (4) the explicit column alignment against `model_meta.json`, which prevents training–serving skew, and (5) logging every prediction with its model run.

Two details avoid classic bugs: the ModelRun created from the metadata (training runs before the database exists), and `Literal` types for `contract_type` and `payment_method`, so that an unknown or misspelled category returns 422 instead of a silently wrong prediction. In the plan review, check that the agent loads the model with XGBoost's `load_model` (not pickle), that the model is loaded once and not per request, and that predictions are written to the database.

Test in `/docs`: call `/api/sample-customer`, copy the JSON into `/api/predict`, then check `/api/predictions`.

POC 3: prompt 3 – frontend

Prompt for Copilot – Agent

In `poc3_xgboost_churn/`, create `frontend/app.py` (Streamlit, calls the backend only via HTTP, `BACKEND_URL` as in POC 2) with three tabs:

Customers: table from GET `/api/customers` with a filter by `contract_type`.

Prediction: a form with all features and sensible defaults, a button "Load sample customer" (GET `/api/sample-customer` fills the form) and a selectbox "Choose existing customer" (POST `/api/predict/{customer_id}`). The button "Predict" calls POST `/api/predict` and shows the probability as a metric and a progress bar.

History: the latest predictions from GET `/api/predictions`, plus the model metrics from GET `/api/model/info`.

Add streamlit and requests to `requirements.txt` and install them. Done when all three tabs work with the sample customer.

One detail goes beyond a plain prediction form: the History tab also shows the model metrics, so users see how good the model is next to its predictions – a small step towards transparency.

The "Load sample customer" button uses the testing endpoint, so that students can try the app without typing values. `st.tabs`, `st.metric` and `st.progress` are standard Streamlit elements; the progress bar takes a value between 0.0 and 1.0 – exactly a probability.

Ask students to try extreme profiles (new customer, monthly contract, many support calls vs. long-term two-year contract) and check whether the probabilities move in the expected direction.

POC 3: run and verify

Sanity checks for an ML app

Model

- ☑ ROC-AUC clearly above 0.5; recall reported
- ☑ model beats the "always no churn" baseline
- ☑ `/api/model/info` shows the metrics

Behaviour

- ☑ high-risk profile → high probability
- ☑ loyal two-year customer → low probability

System

- ☑ every prediction appears in the history
- ☑ invalid input in `/docs` (text as age) → 422, no crash
- ☑ restart: model and data still there

Too good to be true?

ROC-AUC near 1.0 on synthetic data usually means the generator made churn too predictable.

POC 3: run and verify

These checks combine the three levels of an ML application: model quality, plausible behaviour, and system behaviour. The behaviour checks are a simple form of what practitioners call "invariance" or "directional" tests: changing a feature in a known direction should move the prediction in the expected direction.

Invalid input should never crash the backend: Pydantic returns 422 with a clear message. Persistence: after a restart, the model is loaded again from `model.json`, and the history is still in SQLite.

The warning box addresses a typical surprise with synthetic data: if the generator uses deterministic rules, the model can learn them perfectly. Suggest adding noise to the generator – and discuss why real churn is much harder to predict.

From prediction to action

A score is not a decision

Who acts?

The model ranks; people decide whom to contact and with which offer. Define the process around the score.

Fair & explainable?

Would the model treat groups unfairly? Can we explain a score to a manager or a customer?

Data protection

Real customer data is personal data: purpose limitation, access control, deletion – GDPR applies.

Automation creates value only when the prediction is embedded in a responsible business process.

From prediction to action

Close the technical part of POC 3 with the business perspective. A churn score is useful only if someone acts on it: which threshold triggers a call, who makes the call, which offer is allowed, how success is measured (e.g. with a control group).

Fairness and explainability: our model even uses age directly – a characteristic protected under the German General Equal Treatment Act (AGG). Ask the students whether it should be used at all. Features that correlate with protected characteristics can disadvantage groups too; feature importance and individual explanations help to detect this. For customer-facing decisions, companies must be able to explain the logic.

Data protection: our data is synthetic. With real customer data, the GDPR principles (purpose limitation, data minimisation, access control, deletion) and internal policies apply – a POC with real data therefore needs the data-protection officer early. The LLM deck returns to governance and the EU AI Act.

POC 3: extensions

EASY **Quick wins**

- feature-importance chart in the History tab
- threshold slider: who gets called?
- colour the probability (green / amber / red)

HARDER **Closer to reality**

- batch scoring: upload a CSV, download scores
- compare two model runs before switching
- train on the IBM Telco churn dataset

Batch scoring is how churn models are used in practice: a nightly list for the sales team.

POC 3: extensions

The extensions move the POC towards real usage. The threshold slider makes the business trade-off tangible: moving it shows how many customers would be contacted and how many churners would be caught (on the test data).

Batch scoring – upload a customer list, get scores back – is the typical production pattern for churn models: a scheduled job scores all customers overnight and hands a prioritised list to sales or marketing. Combined with an automation tool, that list could trigger retention workflows automatically.

Comparing model runs before switching introduces the idea of model governance: a new model is only deployed if it is demonstrably better on the same test data.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

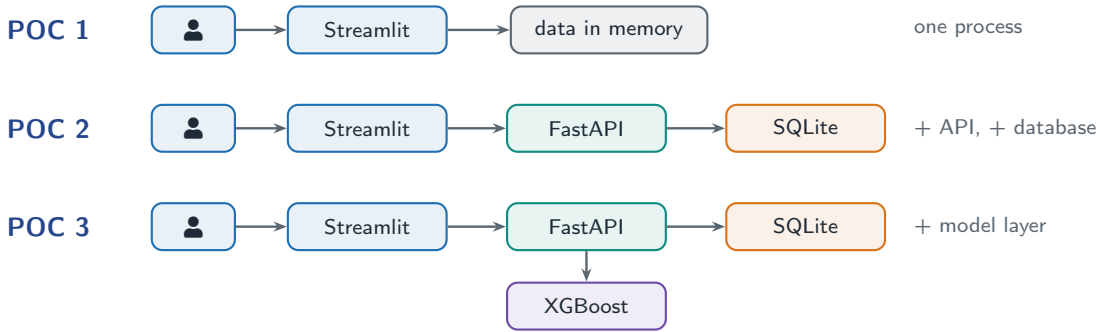
Part 8 • Session 2 and following: Wrap-up

What you built, what is still missing – and what comes next.

- ✔ Compare the three architectures at a glance
- ✔ Know what separates a POC from a product
- ✔ Take seven habits into every future project

Outline

Three POCs at a glance



Each stage *adds* to the architecture – nothing is taken away.

Three POCs at a glance

Use this summary diagram to let the students retell the journey in their own words. POC 1: one process, everything in memory. POC 2: the same functionality, but with an API and a database – persistence, reuse, separation. POC 3: the same architecture plus a model layer – data, model, API and UI together form a small ML product.

Question for the audience: "Which of the three would you choose for (a) a one-off data exploration, (b) an internal tool used by ten colleagues, (c) a churn score that the CRM system should use every night?" Answers: (a) POC 1, (b) POC 2, (c) POC 3 – and for (c) the API would be called by the CRM or an automation workflow, not only by Streamlit.

From POC to production

What we deliberately left out – but a product needs

Topic	What a product needs	Who usually owns it
Tests	automated tests for logic and API	development team
Access	login, roles, permissions	IT / security
Deployment	servers or cloud, backups, updates	IT operations
Monitoring	errors, performance, model drift	operations / data science
Data protection	GDPR, data minimisation, deletion	data-protection officer
Ownership	who maintains and pays for it?	business owner

A POC answers *whether* it works. Production answers *how* it keeps working – safely.

Return to the question from Part 1: what would be missing before the finance department could use the expense-claim checker with real data? This table is the structured answer. Each row is a whole discipline, which is why POCs should be handed over to professionals once the idea is proven – or rebuilt properly.

Model drift deserves a sentence: the churn model was trained on past data; if customer behaviour changes (new competitor, new tariffs), its predictions get worse without any error message. Monitoring the model's performance over time is therefore part of operating an ML application.

The "ownership" row is the most underestimated one in practice: many good prototypes die because nobody is responsible for them after the project ends.

Seven take-aways

1. **POC = learning speed over robustness.** Three small apps beat one perfect project.
2. **AI-assisted coding is a loop.** Describe, generate, review, test, commit.
3. **Prompt quality = code quality.** Goal, stack, files, interfaces, demo data, done-criteria.
4. **Read it, run it, check it.** Only then commit it.
5. **Architecture is "who calls whom, how"** – the use case can stay the same.
6. **ML app = three tiers + a model layer** – and the right metrics.
7. **Secrets never go into Git.** `.gitignore` before the first commit.

You now own a professional toolchain – and three prototypes that prove you can use it.

Seven take-aways

Summarise the seven habits. Invite students to pick the one they found most surprising – typically "prompt quality equals code quality" or "architecture is who calls whom".

Link to the next deck: everything the Copilot agent did in these sessions – planning, calling tools, reading results, iterating – is an instance of agentic AI. The LLM deck explains how large language models, retrieval-augmented generation and agents work, and uses the same AI-assisted coding method for three LLM prototypes (chat with your PDF, semantic product search, a ShopCo support agent).

Recommended follow-up task: write a one-page reflection on one POC – what worked, what the agent got wrong, and how you noticed.

Resources

Course repository

github.com/ChrisW09/Python-for-AI-Driven-Automation
From your first line of Python to AI-driven automation: notebooks, exercises, data and slides.

Documentation

VS Code AI	code.visualstudio.com/docs
GitHub Copilot	docs.github.com/copilot
Streamlit	docs.streamlit.io
FastAPI	fastapi.tiangolo.com
SQLAlchemy	docs.sqlalchemy.org
XGBoost	xgboost.readthedocs.io
Git book	git-scm.com/book

Further reading

Simon Willison's blog on AI-assisted programming: simonwillison.net

MDN Web Docs on HTTP: developer.mozilla.org

Outlook: React, Vue, Svelte – frontend frameworks for products, not used here.

Resources

Point to the course repository first: it is the home of the module's materials – Jupyter notebooks from Python basics to AI engineering, exercises with solutions, sample data and the slides. Students who are new to Python start with the onboarding and foundations modules. All documentation links are official sources and were checked in early October 2026. The VS Code and GitHub Copilot documentation changes frequently – whenever the students' screens differ from the slides, the docs are the reference.

The Git book (Pro Git, by Scott Chacon and Ben Straub) is available free online, also in German (`git-scm.com/book/de`).

Simon Willison's blog is one of the most careful practitioner sources on using LLMs for programming, including the distinction between vibe coding and responsible AI-assisted development that we used in Part 1.

References

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv:2507.09089.

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proc. KDD '16*, 785–794.

Dell'Acqua, F., et al. (2026). Navigating the jagged technological frontier: Field experimental evidence of the effects of artificial intelligence on knowledge worker productivity and quality. *Organization Science* 37(2), 403–423.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. PhD dissertation, University of California, Irvine.

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS 35* (Datasets & Benchmarks).

Karpathy, A. (2025, 2 Feb). "There's a new kind of coding I call 'vibe coding' ..." [Post on X]. https://x.com/karpathy/status/1886192184808149383

Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science* 381(6654), 187–192.

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv:2302.06590.

Willison, S. (2025, 19 Mar). Not all AI-assisted programming is vibe coding (but vibe coding rocks). https://simonwillison.net/2025/Mar/19/vibe-coding/

All references were checked against the primary sources in September 2026. Note on Dell'Acqua et al.: the study circulated as Harvard Business School Working Paper 24-013 (2023) and was published in Organization Science in 2026; the published version reports a quality improvement of about 32% inside the frontier, lower than the "more than 40%" often quoted from the working paper.

Product documentation used for facts about GitHub Copilot, VS Code, Python, Streamlit, FastAPI, SQLite, SQLAlchemy and XGBoost is listed on the resources slide; these facts reflect the state of September 2026.

Questions?

Next: LLMs, RAG & Agentic AI — the building blocks of AI-driven automation

christoph.weisser@hsbi.de
github.com/ChrisW09/Python-for-AI-Driven-Automation

Prof. Dr. Christoph Weisser • AI-Driven Automation • Bielefeld School of Business, HSBI

Open the floor for questions. Useful prompts if the room is quiet: "Where did the agent surprise you – positively or negatively?", "Which business process from your internship would you prototype next?"

Point once more to the course repository for practice material. Remind students to push their final POC versions to GitHub before the next session and to keep their Copilot setup ready for the LLM prototypes.